

C 语言题库

使用说明

1. 建议按章顺序练习；也可按薄弱主题跳章。
2. 先独立完成「一、题目」，再对照「二、参考答案与解析」。
3. 「三、本章知识点总结」用于考前速记，勿跳过。
4. 读程序题默认在常见 64 位 GCC/MSVC 环境下验证；`sizeof` 等若有差异已在答案中注明。
5. 全书约 420 题（每章约 30 题），分 14 章。

目录

1. [程序基础](#)
2. [数据类型与常量](#)
3. [运算符](#)
4. [流程控制](#)
5. [函数](#)
6. [指针](#)
7. [数组](#)
8. [字符串](#)
9. [结构体与枚举](#)
10. [预处理](#)
11. [输入输出](#)
12. [文件操作](#)
13. [动态内存](#)
14. [常用库函数](#)

第 1 章 · 程序基础

本章知识点速览

- 编译四阶段：预处理 → 编译 → 汇编 → 链接
- `main` 是程序入口，通常 `return 0` 表示正常结束
- 标识符：字母/下划线开头，区分大小写，不能是关键字
- `.c` 放实现，`.h` 放声明/宏/类型；`#include` 引入头文件
- 注释：`//` (C99) 与 `/* */` (不可嵌套)

一、题目

选择题

1. 下列关于 C 语言编译流程的说法，正确的是：
 - A. 预处理直接生成可执行文件
 - B. 链接阶段处理 `#include` 与 `#define`
 - C. 编译阶段将 C 代码翻译为汇编代码
 - D. 汇编阶段合并目标文件与库

2. 程序的入口函数通常是:
- A. `start`
 - B. `main`
 - C. `init`
 - D. `printf`
3. 下列哪个是合法的标识符?
- A. `2count`
 - B. `my-var`
 - C. `_total`
 - D. `int`
4. 关于 `.c` 与 `.h` 文件, 正确的是:
- A. `.h` 中通常写函数的具体实现
 - B. `.c` 中通常写宏定义与类型声明供他人包含
 - C. `.c` 存放实现, `.h` 存放声明, 由 `#include` 引入
 - D. 两者完全等价, 可随意互换
5. 下列关于注释的说法, 错误的是:
- A. `//` 单行注释从 C99 起支持
 - B. `/* ... */` 可以嵌套使用
 - C. `/* ... */` 可用于多行注释
 - D. 注释内容不参与编译后的可执行逻辑
6. 下列哪个是 C 语言关键字?
- A. `main`
 - B. `printf`
 - C. `sizeof`
 - D. `Hello`
7. `gcc -c hello.c` 的作用是:
- A. 编译并链接生成可执行文件
 - B. 只预处理不编译
 - C. 只编译不链接, 生成目标文件
 - D. 直接运行程序
8. 预处理阶段的常见产物扩展名是:
- A. `.s`
 - B. `.i`
 - C. `.o`
 - D. `.exe`
9. 下列关于 C 语言特点的说法, 错误的是:
- A. 面向过程, 以函数为组织单位
 - B. 编译型语言, 先编译再运行
 - C. 自动管理全部动态内存, 无需程序员释放
 - D. 可通过指针贴近硬件操作
10. `#include <stdio.h>` 中尖括号表示:
- A. 优先在当前目录查找头文件
 - B. 在系统/标准头文件搜索路径中查找
 - C. 只用于宏定义
 - D. 与 `#include "stdio.h"` 完全等价且无差别
11. 下列哪个不是合法标识符?
- A. `student_name`
 - B. `_123`

- C. goto
- D. MaxScore

12. 语句与表达式的区别，正确的是：
- A. 表达式必须以分号结尾
 - B. 语句通常以分号结尾，表达式是有值的式子
 - C. 两者完全相同
 - D. 复合语句不能用 {} 包裹

填空题

13. 编译四阶段依次是：预处理、__、汇编、__。
14. 源文件扩展名通常是 __，头文件扩展名通常是 __。
15. 标识符首字符必须是 __ 或 __。
16. int main(void) 中，return 0; 一般表示程序 __ 结束。
17. gcc hello.c -o hello 中，-o hello 的作用是指定 __。
18. 用 {} 包裹的多条语句称为 __ 语句。
19. C 语言区分大小写，因此变量 Sum 与 sum 是 __ 个不同的标识符。

读程序写结果

20. 写出程序输出：

```
#include <stdio.h>
int main(void) {
    printf("Hello, C!\n");
    return 0;
}
```

21. 写出程序输出：

```
#include <stdio.h>
int main(void) {
    int Sum = 3;
    int sum = 5;
    printf("%d %d\n", Sum, sum);
    return 0;
}
```

22. 写出程序输出：

```
#include <stdio.h>
int main(void) {
    /* printf("A"); */
    // printf("B");
    printf("C");
    return 0;
}
```

23. 写出程序输出：

```
#include <stdio.h>
int main(void) {
    printf("%d\n", 1 + 2 * 3);
    printf("OK\n");
    return 0;
}
```

24. 写出程序输出：

```
#include <stdio.h>
int main(void) {
    int a = 10;
    {
        int a = 20;
        printf("%d ", a);
    }
    printf("%d\n", a);
    return 0;
}
```

25. 写出程序输出：

```
#include <stdio.h>
#define MSG "C"
int main(void) {
    printf("%s Language\n", MSG);
    return 0;
}
```

简答或编程

26. 简述预处理、编译、汇编、链接四个阶段各自的主要工作，并写出各阶段常见产物扩展名。
27. 说明为什么不能把关键字（如 `int`、`return`）当作变量名使用；并举一个合法改写示例。
28. 写出一个最小的可运行 C 程序，要求输出一行 `Hi`（含换行）。
29. 说明 `#include "my.h"` 与 `#include <stdio.h>` 在查找路径习惯上的主要区别。
30. 简述为什么要把函数声明放在 `.h`、实现放在 `.c`（至少两点好处）。

二、参考答案与解析

1. 答案：C

解析：预处理处理 `#` 指令；编译生成汇编；汇编生成目标文件；链接生成可执行文件。

2. 答案：B

解析：C 程序从 `main` 开始执行。

3. 答案：C

解析：`_total` 合法；不能以数字开头，不能含 `-`，不能是关键字。

4. 答案：C

解析：约定是实现与声明分离：`.c` 实现，`.h` 声明。

5. 答案：B

解析：`/* */` 注释不能嵌套。

6. 答案：C

解析：`sizeof` 是关键字；`main`、`printf` 是标识符/库函数名。

7. 答案：C

解析：`-c` 表示只编译到目标文件（`.o` / `.obj`），不链接。

8. 答案：B

解析：预处理产物常为 `.i`；编译 `.s`；汇编 `.o`；链接可执行文件。

9. 答案：C

解析：C 无自动垃圾回收，`malloc` 等需手动 `free`。

10. 答案：B

解析：尖括号通常搜系统路径；双引号通常先搜当前目录。

11. 答案：C

解析：`goto` 是关键字，不能作标识符。

12. 答案：B

解析：表达式有值；语句执行动作，常以 `;` 结束。

13. 答案：编译；链接

解析：四阶段固定顺序：预处理→编译→汇编→链接。

14. 答案：`.c`；`.h`

解析：源文件与头文件扩展名约定。

15. 答案：字母；下划线 `_`

解析：首字符不能是数字。

16. 答案：正常（成功）

解析：返回 0 通常表示成功，非 0 表示异常。

17. 答案：输出文件名（可执行文件名）

解析：`-o` 指定生成目标名称。

18. 答案：复合

解析：复合语句用 `{}` 组成语句块。

19. 答案：两

解析：大小写敏感。

20. 答案：

```
Hello, C!
```

解析：标准最小程序输出，`\n` 换行。

21. 答案：

```
3 5
```

解析：标识符区分大小写，`Sum` 与 `sum` 是两个变量。

22. 答案：

```
C
```

解析：前两行输出被注释掉，只打印 `C`。

23. 答案：

```
7
OK
```

解析：先乘后加得 7，再输出 `OK`。

24. 答案：

```
20 10
```

解析：内层块的 `a` 遮蔽外层；离开块后外层仍为 10。

25. 答案：

```
C Language
```

解析：预处理用 `MSG` 替换为 `C`。

26. 答案要点：

预处理：处理 `#include` / `#define` 等，常见产物 `.i`；编译：C→汇编，`.s`；汇编：汇编→机器码目标文件，`.o` / `.obj`；链接：合并目标文件与库，生成可执行文件。

27. 答案要点：

关键字由语言保留，有固定语法含义，用作变量名会导致语法错误。示例：用 `my_int`、`ret_val` 等代替。

28. 参考代码：

```
#include <stdio.h>
int main(void) {
    printf("Hi\n");
    return 0;
}
```

29. 答案要点:

"my.h" 通常先在当前/指定目录查找; <stdio.h> 在系统标准头文件路径查找。

30. 答案要点:

接口与实现分离, 便于多文件共享声明、隐藏实现、减少重复定义与链接冲突, 提高可维护性。

三、本章知识点总结

要点	速记
编译流程	预→编→汇→链
入口	main, return 0 成功
标识符	字母/_ 开头, 区分大小写
文件	.c 实现 / .h 声明
注释	// 与 /* */, 后者不可嵌套

第 2 章 · 数据类型与常量

本章知识点速览

- 基本类型: char/short/int/long/long long/float/double/void
- 常见 64 位: sizeof(int)=4, sizeof(double)=8, sizeof(char)=1
- 进制: 八进制前缀 0, 十六进制 0x; 字符 'A', 字符串 "hi" (含 \0)
- signed/unsigned、const; 局部未初始化值不确定, 静态/全局默认为 0
- 转义字符与隐式/显式类型转换

一、题目

选择题

- 在常见 64 位环境下, sizeof(int) 的值通常是:
A. 1
B. 2
C. 4
D. 8
- 整型常量 0xFF 的十进制值是:
A. 255

- B. 15
 - C. 256
 - D. 100
3. 下列关于 `unsigned int` 的说法，正确的是：
- A. 可以表示负数
 - B. 只表示非负整数
 - C. 与 `int` 取值范围完全相同
 - D. 不能参与运算
4. `int x = 3.9;` 执行后 `x` 的值是：
- A. 3.9
 - B. 4
 - C. 3
 - D. 不确定
5. 下列哪一项是字符常量？
- A. `"A"`
 - B. `'A'`
 - C. `A`
 - D. `\A`
6. 局部变量 `int a;` 未初始化时：
- A. 一定是 0
 - B. 一定是 1
 - C. 值不确定
 - D. 编译报错
7. 转义字符 `\n` 的含义是：
- A. 制表符
 - B. 反斜杠
 - C. 换行
 - D. 空字符
8. 在常见 64 位环境下，`sizeof(float)` 通常是：
- A. 2
 - B. 4
 - C. 8
 - D. 16
9. 下列关于 `const int N = 10;` 的说法，正确的是：
- A. `N` 可以再次赋值修改
 - B. `N` 只读，有类型，受编译器检查
 - C. 与 `#define N 10` 完全等价、无任何区别
 - D. `N` 只能是全局变量
10. 全局变量 `int g;` 未显式初始化时，其值通常是：
- A. 不确定
 - B. 1
 - C. 0
 - D. 随机但每次相同
11. 字符串常量 `"A"` 与字符常量 `'A'` 的主要区别是：
- A. 两者完全相同
 - B. `"A"` 含结束符，占更多字节；`'A'` 是单个字符
 - C. `'A'` 一定是字符串
 - D. `"A"` 不能用于初始化字符数组

12. `1.2e-2` 的值是：

- A. 1.2
- B. 0.012
- C. 12
- D. 120

填空题

- 13. 八进制整型常量以数字 `_` 开头；十六进制以 `_` 开头。
- 14. 字符串 `"Hi"` 在内存中实际包含 `_` 个字符（含结束符），结束符是 `_`。
- 15. `sizeof(char)` 的值是 `_`；常见 64 位下 `sizeof(double)` 是 `_`。
- 16. 强制把 `3.9` 转为整型可写作 `_`，结果为 `_`。
- 17. 转义字符 `\t` 表示 `_`；`\0` 表示 `_`。
- 18. 单精度浮点常量常加后缀 `_`，如 `3.14f`。
- 19. `short` 典型大小为 `_` 字节；`long long` 典型大小为 `_` 字节。

读程序写结果

20. 写出程序输出（常见 64 位）：

```
#include <stdio.h>
int main(void) {
    printf("%zu %zu\n", sizeof(int), sizeof(char));
    return 0;
}
```

21. 写出程序输出：

```
#include <stdio.h>
int main(void) {
    int a = 0377;
    int b = 0xFF;
    printf("%d %d\n", a, b);
    return 0;
}
```

22. 写出程序输出：

```
#include <stdio.h>
int main(void) {
    char ch = 'A';
    printf("%c %d\n", ch, ch);
    printf("%d\n", (int)3.9);
    return 0;
}
```

23. 写出程序输出：

```
#include <stdio.h>
int main(void) {
    int x = 'B' - 'A';
    printf("%d\n", x);
    printf("%c\n", 'A' + 2);
    return 0;
}
```

24. 写出程序输出（常见 64 位）：

```
#include <stdio.h>
int main(void) {
    printf("%zu %zu\n", sizeof(float), sizeof(long long));
    return 0;
}
```

25. 写出程序输出：

```
#include <stdio.h>
int g;
int main(void) {
    static int s;
    int a = 5;
    printf("%d %d %d\n", g, s, a);
    return 0;
}
```

简答或编程

26. 比较 `#define PI 3.14` 与 `const double PI = 3.14;` 的主要区别。
27. 说明 `signed` 与 `unsigned` 的含义，并指出为什么无符号类型更适合表示“数量/长度”。
28. 列举至少 4 种常用转义字符及其含义。
29. 说明隐式类型转换与显式强制转换的区别，并各举一例。
30. 写出声明：一个不可修改的整型常量 `MAX`，值为 100；再写出一个无符号整型变量 `len` 并初始化为 0。

二、参考答案与解析

1. 答案：C

解析：常见 32/64 位桌面环境下 `int` 多为 4 字节。

2. 答案：A

解析：`0xFF = 15*16+15 = 255`。

3. 答案：B

解析：`unsigned` 只表示非负数，范围从 0 起。

4. 答案：C

解析：浮点赋给整型发生隐式截断，小数部分丢弃。

5. 答案: B

解析: 单引号表示字符常量; 双引号是字符串。

6. 答案: C

解析: 自动存储期局部变量未初始化则值不确定。

7. 答案: C

解析: `\n` 换行, `\t` 制表, `\\` 反斜杠, `\0` 空字符。

8. 答案: B

解析: `float` 常见 4 字节, `double` 常见 8 字节。

9. 答案: B

解析: `const` 有类型且只读; `#define` 是文本替换。

10. 答案: C

解析: 静态存储期对象未初始化默认为 0。

11. 答案: B

解析: `"A"` 实际是 `'A'` 与 `'\0'`; `'A'` 是单个字符值。

12. 答案: B

解析: $1.2 \times 10^{(-2)} = 0.012$ 。

13. 答案: 0; 0x (或 0X)

解析: 如 `0377`、`0xFF`。

14. 答案: 3; `\0`

解析: `'H'`、`'i'`、`'\0'`。

15. 答案: 1; 8

解析: `char` 恒为 1; `double` 常见为 8。

16. 答案: `(int)3.9`; 3

解析: 显式强制转换, 向零截断。

17. 答案: 制表符; 空字符 (字符串结束符)

解析: 常见转义必记。

18. 答案: `f` 或 `F`

解析: 否则字面量默认为 `double`。

19. 答案: 2; 8

解析: 典型 LP64/常见桌面环境取值。

20. 答案:

解析：常见 64 位下 `int` 4 字节，`char` 1 字节。

21. 答案：

255 255

解析：`0377` 与 `0xFF` 均为 255。

22. 答案：

A 65
3

解析：`'A'` ASCII 为 65；`(int)3.9` 截断为 3。

23. 答案：

1
C

解析：相邻字符码差 1；`'A'+2` 为 `'C'`。

24. 答案：

4 8

解析：`float` 4；`long long` 8。

25. 答案：

0 0 5

解析：全局与静态局部默认 0；自动局部已初始化为 5。

26. 答案要点：

`#define` 为预处理文本替换，无类型；`const` 有类型、受编译器类型检查，通常更安全。

27. 答案要点：

`signed` 可负可正；`unsigned` 仅非负。长度/个数不会为负，用无符号可扩大正数范围并表达语义。

28. 答案要点：

`\n` 换行；`\t` 制表；`\\` 反斜杠；`\'` 单引号；`\"` 双引号；`\0` 空字符等。

29. 答案要点:

隐式：编译器自动转换，如 `int x = 3.9;` 截断为 3。显式：程序员写 `(type)`，如 `(int)3.9`。

30. 参考写法:

```
const int MAX = 100;
unsigned int len = 0;
```

三、本章知识点总结

类型/写法	要点
<code>int</code> / <code>double</code>	常见 4 / 8 字节
<code>0</code> / <code>0x</code>	八进制 / 十六进制
<code>'A'</code> / <code>"A"</code>	字符 / 字符串 (含 <code>\0</code>)
初始化	局部未初始化不定；静态/全局默 0
转换	隐式自动；显式 <code>(type)</code>

第 3 章 · 运算符

本章知识点速览

- 整数除法截断: `5/2=2`；浮点参与则得浮点结果
- 前缀/后缀自增: `++i` 先加后用, `i++` 先用后加
- 逻辑短路: `&&` 左假不算右, `||` 左真不算右
- 位运算: `& | ^ ~ << >>`；注意与逻辑运算区分
- 优先级: 单目→算术→移位→关系→位→逻辑→条件→赋值; `==` 与 `=` 易混

一、题目

选择题

1. 表达式 `5 / 2` 的值是:
A. 2.5
B. 2
C. 3
D. 2.0
2. 设 `int i = 3;`, 则表达式 `i++` 的值与之后 `i` 分别为:
A. 4 和 4
B. 3 和 4
C. 4 和 3
D. 3 和 3

3. 关于 `&&` 短路，下列正确的是：
- A. 左边为真才不算右边
 - B. 左边为假则不算右边
 - C. 两边一定都会计算
 - D. 只用于位运算
4. `int a = 5, b = 3;` 则 `a & b` 的结果是：
- A. 1
 - B. 7
 - C. 6
 - D. 0
5. 下列优先级最高的是：
- A. `+`
 - B. `*` (乘法)
 - C. `=`
 - D. `&&`
6. `if (x = 0)` 与 `if (x == 0)` 的区别是：
- A. 两者完全相同
 - B. 前者赋值，条件恒为假；后者比较是否等于 0
 - C. 前者比较，后者赋值
 - D. 两者都会导致编译错误
7. `a > b ? a : b` 的含义是：
- A. 求 a 与 b 的和
 - B. 若 $a > b$ 取 a，否则取 b
 - C. 位运算
 - D. 强制类型转换
8. 表达式 `!0` 的值是：
- A. 0
 - B. 1
 - C. -1
 - D. 不确定
9. `a += 3` 等价于：
- A. `a = 3`
 - B. `a = a + 3`
 - C. `a == a + 3`
 - D. `a + 3`
10. 关于逗号运算符，正确的是：
- A. 优先级最高
 - B. 整个表达式的值取最后一个子表达式的值
 - C. 与赋值优先级相同
 - D. 不能出现在 `for` 中
11. `int n = 7;` 则 `n % 2 == 0` 为真还是假？
- A. 真
 - B. 假
 - C. 编译错误
 - D. 不确定
12. `4 | 1` 的结果是：
- A. 0
 - B. 4

C. 5

D. 1

填空题

13. `5.0 / 2` 的结果是 __; `5 % 2` 的结果是 __。
14. 设 `int i = 1;`, 执行 `int x = ++i;` 后, `x` 为 __, `i` 为 __。
15. 表达式 `0 && (1/0)` 不会触发除零, 原因是 __。
16. `a << 2` 在忽略溢出时约相当于把 `a` 乘以 __。
17. 关系运算符比较结果为真时值为 __, 为假时值为 __。
18. 赋值运算符结合性是从 __ 到 __。
19. `a ^ a` 的结果通常是 __。

读程序写结果

20. 写出程序输出:

```
#include <stdio.h>
int main(void) {
    int a = 5, b = 2;
    printf("%d %d\n", a / b, a % b);
    printf("%.1f\n", a / 2.0);
    return 0;
}
```

21. 写出程序输出:

```
#include <stdio.h>
int main(void) {
    int i = 1, j = 1;
    int a = i++;
    int b = ++j;
    printf("%d %d %d %d\n", a, i, b, j);
    return 0;
}
```

22. 写出程序输出:

```
#include <stdio.h>
int main(void) {
    int x = 0, y = 5;
    int r1 = x && (++y);
    int r2 = x || (++y);
    printf("%d %d %d\n", r1, r2, y);
    return 0;
}
```

23. 写出程序输出:

```
#include <stdio.h>
int main(void) {
    int a = 6, b = 3;
    printf("%d %d %d\n", a & b, a | b, a ^ b);
    return 0;
}
```

24. 写出程序输出：

```
#include <stdio.h>
int main(void) {
    int a = 3, b = 5;
    int m = a > b ? a : b;
    printf("%d\n", m);
    a *= 2;
    printf("%d\n", a);
    return 0;
}
```

25. 写出程序输出：

```
#include <stdio.h>
int main(void) {
    int x = (1, 2, 3);
    int y = 8 >> 1;
    printf("%d %d\n", x, y);
    return 0;
}
```

简答或编程

26. 解释为什么写条件判断时推荐 `if (x == 0)` 而不是 `if (x = 0)`。
27. 用一句话说明运算符优先级记忆口诀，并指出逗号运算符相对赋值的优先级高低。
28. 写出判断闰年的条件表达式（年份变量为 `y`）。
29. 说明逻辑运算符 `&&/||` 与位运算符 `&/|` 的两点重要区别。
30. 用条件运算符写一句：把 `x` 赋值为其绝对值（不调用库函数）。

二、参考答案与解析

1. 答案：B

解析：两个整数相除结果仍为整数，向零截断。

2. 答案：B

解析：后缀自增表达式值为加之前的值，随后 `i` 变为 4。

3. 答案：B

解析：`&&` 左操作数为假即可确定整体为假，右边不计算。

4. 答案：A

解析：5 为 101，3 为 011，按位与得 001，即 1。

5. 答案：B

解析：乘法高于加、逻辑与、赋值。

6. 答案：B

解析：= 赋值，x=0 表达式值为 0（假）；== 才是相等比较。

7. 答案：B

解析：条件运算符：条件真取第二项，假取第三项。

8. 答案：B

解析：逻辑非：0 的非为 1，非 0 的非为 0。

9. 答案：B

解析：复合赋值 a += 3 即 a = a + 3。

10. 答案：B

解析：逗号运算符求值从左到右，取最后一项；优先级最低。

11. 答案：B

解析：7 是奇数，7 % 2 == 1，条件为假。

12. 答案：C

解析：100 | 001 = 101，即 5。

13. 答案：2.5；1

解析：有浮点则浮点除；% 为取余。

14. 答案：2；2

解析：前缀先加，表达式值与变量均为加后值。

15. 答案：逻辑与短路，左边为假不再计算右边

解析：短路求值是重要考点。

16. 答案：4

解析：左移 1 位 $\times 2$ ，左移 2 位 $\times 4$ 。

17. 答案：1；0

解析：C 中关系/逻辑结果用 1/0 表示。

18. 答案：右；左

解析：如 a = b = 5 先给 b 赋值。

19. 答案：0

解析：异或相同位得 0。

20. 答案：

2 1
2.5

解析：整数除/取余；`2.0` 触发浮点除。

21. 答案：

1 2 2 2

解析：`a=i++` 得 1 后 $i=2$ ；`b=++j` 先加得 2。

22. 答案：

0 1 6

解析：`x&&(++y)` 因 $x=0$ 短路， y 仍 5；`x||(++y)` 会算右边， y 变为 6，结果为真 1。

23. 答案：

2 7 5

解析：`6=110`，`3=011`；与=010(2)，或=111(7)，异或=101(5)。

24. 答案：

5
6

解析：三目取较大者 5；`a\*=2` 后 a 从 3 变为 6。

25. 答案：

3 4

解析：逗号取最后的 3；`8>>1` 得 4。

26. 答案要点：

`=` 是赋值，容易误写成恒假/恒真条件且改变变量；`==` 才是比较。有的风格写 `if (0 == x)` 可让误写 `=` 更易被编译器发现。

27. 答案要点：

口诀：单目→算术→移位→关系→位运算→逻辑→条件→赋值→逗号。逗号优先级低于赋值。

28. 参考表达式：

```
(y % 4 == 0 && y % 100 != 0) || (y % 400 == 0)
```

29. 答案要点：

① 逻辑运算按“真/假”且有短路；位运算按二进制位计算且无短路。② 结果语义不同：逻辑得 0/1，位运算得按位组合后的整数。

30. 参考写法：

```
x = x < 0 ? -x : x;
```

三、本章知识点总结

- 整除截断；要小数就让一侧为浮点
- `++i` / `i++`：先加后用 / 先用后加
- `&&` `||` 有短路；`&` `|` 是位运算、无短路
- `==` 比较，`=` 赋值，条件里别写错
- 优先级拿不准就加括号

第 4 章 · 流程控制

本章知识点速览

- `if-else` 做范围/条件分支；`switch` 做整型等值多路分支（注意 `break`）
- `for` 适合已知次数；`while` 先判后执；`do-while` 至少执行一次
- `break` 跳出当前循环/ `switch`；`continue` 结束本轮进入下一轮
- 多条语句作分支/循环体必须用 `{ }`

一、题目

选择题

1. `switch` 的 case 标签后面应当是：
A. 变量
B. 整型常量表达式
C. 任意表达式
D. 浮点常量
2. 关于 `do-while`，正确的是：
A. 可能一次也不执行
B. 至少执行一次
C. 不能使用 `break`
D. 条件写在循环体开头
3. `for (int i = 0; i < 3; i++)` 循环体执行次数是：
A. 2
B. 3
C. 4
D. 0

4. `continue` 的作用是:
- A. 结束整个程序
 - B. 跳出所有嵌套循环
 - C. 结束本次循环, 进入下一次
 - D. 与 `break` 完全相同
5. 下列循环可能一次也不执行的是:
- A. 仅 `do-while`
 - B. `for` 与 `while`
 - C. 仅 `for`
 - D. 三种都至少一次
6. `switch` 中某个 `case` 缺少 `break` 时:
- A. 编译错误
 - B. 自动跳到 `default` 结束
 - C. 可能落入后续 `case` 继续执行 (贯穿)
 - D. 忽略该 `case`
7. `if (score >= 60) printf("pass"); else printf("fail");` 当 `score=60` 时输出:
- A. fail
 - B. pass
 - C. 无输出
 - D. 编译错误
8. `switch` 的控制表达式通常不能是:
- A. `int`
 - B. `char`
 - C. `enum` 常量对应的整型
 - D. `double`
9. 关于 `break`, 错误的是:
- A. 可跳出当前循环
 - B. 可跳出当前 `switch`
 - C. 一次可跳出全部嵌套循环
 - D. 只影响最内层当前结构
10. `for (;;)` 的含义通常是:
- A. 语法错误
 - B. 只执行一次
 - C. 死循环 (条件恒真的空缺形式)
 - D. 永不进入循环
11. 多条语句作为 `if` 分支体时, 应当:
- A. 用逗号连接即可
 - B. 用 `{ }` 组成复合语句
 - C. 只能写一条语句, 无法扩展
 - D. 必须写成独立函数
12. `while (0) { printf("A"); }` 的输出是:
- A. A
 - B. 无限输出 A
 - C. 无输出
 - D. 编译错误

填空题

13. `while` 是先 __ 后执行; `do-while` 是先执行后 __。

14. `break` 可跳出当前 __ 或 __。
15. `for` (初始化; 条件; 更新) 中, __ 只在进入循环前执行一次。
16. `do { ... } while` (条件) 末尾的 __ 不能省略。
17. `if-else if` 更适合 __ 判断; `switch` 更适合 __ 分支。
18. `continue` 在 `for` 中跳过后, 接下来会执行 __ 部分再判断条件。
19. `default` 分支在 `switch` 中是 __ 的 (可选/必选) 。

读程序写结果

20. 写出程序输出:

```
#include <stdio.h>
int main(void) {
    int n = 3;
    if (n > 5)
        printf("A");
    else if (n > 2)
        printf("B");
    else
        printf("C");
    return 0;
}
```

21. 写出程序输出:

```
#include <stdio.h>
int main(void) {
    int s = 0;
    for (int i = 1; i <= 5; i++)
        s += i;
    printf("%d\n", s);
    return 0;
}
```

22. 写出程序输出:

```
#include <stdio.h>
int main(void) {
    int i;
    for (i = 1; i <= 5; i++) {
        if (i == 3) continue;
        if (i == 5) break;
        printf("%d ", i);
    }
    printf("\n");
    return 0;
}
```

23. 写出程序输出:

```
#include <stdio.h>
int main(void) {
    int op = 2;
    switch (op) {
        case 1: printf("one ");
        case 2: printf("two ");
        case 3: printf("three ");
        default: printf("end");
    }
    return 0;
}
```

24. 写出程序输出:

```
#include <stdio.h>
int main(void) {
    int i = 0;
    do {
        printf("%d", i);
        i++;
    } while (i < 3);
    printf("\n");
    return 0;
}
```

25. 写出程序输出:

```
#include <stdio.h>
int main(void) {
    int n = 0;
    while (n < 5) {
        n++;
        if (n % 2 == 0) continue;
        printf("%d ", n);
    }
    printf("\n");
    return 0;
}
```

简答或编程

26. 比较 `for`、`while`、`do-while` 三者的判断时机与最少执行次数，并各举一个适用场景。
27. 写出一段小程序：用循环计算 1 到 100 的和并输出（任选一种循环）。
28. 说明为什么 `switch` 的每个 `case` 末尾通常要写 `break`。
29. 编写程序：输入一个 1-7 的整数，用 `switch` 输出对应星期英文缩写（如 1→Mon），其他输出 `Error`。
30. 用嵌套循环打印 3 行，每行输出 `***`（共 3 行星号）。

二、参考答案与解析

1. 答案：B

解析: `case` 后必须是整型常量表达式 (含枚举常量)。

2. 答案: B

解析: 先执行循环体再判断条件, 故至少一次。

3. 答案: B

解析: `i` 取 0、1、2, 共 3 次。

4. 答案: C

解析: `continue` 跳过本轮剩余语句; `break` 才跳出循环。

5. 答案: B

解析: 前测循环在条件一开始为假时执行 0 次。

6. 答案: C

解析: 缺少 `break` 会发生 case 贯穿。

7. 答案: B

解析: `60 >= 60` 为真, 走 `pass` 分支。

8. 答案: D

解析: `switch` 需要整型族, 不能直接用浮点。

9. 答案: C

解析: 一次 `break` 只跳出一层; 多层要用标志或 `goto` 等。

10. 答案: C

解析: 三部分都可省略, 条件空缺视为真, 形成死循环。

11. 答案: B

解析: 否则只有紧随的第一条语句属于分支。

12. 答案: C

解析: 条件一开始为假, 循环体不执行。

13. 答案: 判断; 判断

解析: 前测 / 后测是两者核心区别。

14. 答案: 循环; `switch`

解析: 只跳出最内层当前结构。

15. 答案: 初始化

解析: 条件每次循环前检查; 更新在每次循环体后。

16. 答案: 分号 ;

解析: `while (条件);` 是语法要求。

17. 答案: 范围 (多条件) ; 等值多路

解析：按场景选型。

18. 答案：更新（第三个表达式）

解析：`continue` 仍会走到 `for` 的更新部分。

19. 答案：可选

解析：没有匹配且无 `default` 则什么也不做。

20. 答案：

B

解析：``n=3`` 不大于 5，但大于 2。

21. 答案：

15

解析：`1+2+3+4+5=15`。

22. 答案：

1 2 4

解析：``i==3`` 跳过；``i==5`` 直接 `break`，不打印 5。

23. 答案：

two three end

解析：从 ``case 2`` 贯穿到后续，直到结束。

24. 答案：

012

解析：``do-while`` 输出 0、1、2。

25. 答案：

1 3 5

解析：偶数 ``continue`` 跳过打印，只输出奇数。

26. 答案要点:

`for` / `while` 前判断, 最少 0 次; `do-while` 后判断, 最少 1 次。场景: 已知次数用 `for`; 条件驱动用 `while`; 菜单输入等至少一次用 `do-while`。

27. 参考代码:

```
#include <stdio.h>
int main(void) {
    int sum = 0;
    for (int i = 1; i <= 100; i++)
        sum += i;
    printf("%d\n", sum); /* 5050 */
    return 0;
}
```

28. 答案要点:

防止 case 贯穿: 匹配后继续执行后续 case 的语句, 导致逻辑错误; 需要贯穿时才故意省略。

29. 参考代码:

```
#include <stdio.h>
int main(void) {
    int d;
    scanf("%d", &d);
    switch (d) {
        case 1: printf("Mon\n"); break;
        case 2: printf("Tue\n"); break;
        case 3: printf("Wed\n"); break;
        case 4: printf("Thu\n"); break;
        case 5: printf("Fri\n"); break;
        case 6: printf("Sat\n"); break;
        case 7: printf("Sun\n"); break;
        default: printf("Error\n");
    }
    return 0;
}
```

30. 参考代码:

```
#include <stdio.h>
int main(void) {
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++)
            printf("*");
        printf("\n");
    }
    return 0;
}
```

三、本章知识点总结

结构	要点
if / else if	适合范围判断
switch	等值分支 + 记得 break
for	已知次数
while / do-while	0 次可能 / 至少 1 次
break / continue	跳出 / 跳过本轮

第 5 章 · 函数

本章知识点速览

- 声明（原型）告诉编译器函数存在；定义含函数体
- 默认值传递：改形参不影响实参；要改实参需传地址
- 递归必须有终止条件；static 局部变量保持值跨调用
- 不要返回局部变量的地址；main 返回 0 表示成功

一、题目

选择题

- 关于函数声明与定义，正确的是：
 - 声明必须包含函数体
 - 定义可以没有函数体
 - 声明给出原型，定义给出实现
 - 两者完全一样
- 下列调用后，实参 x 的值会改变的是（x 为 int）：
 - void f(int a) { a = 10; } 调用 f(x);
 - void f(int *a) { *a = 10; } 调用 f(&x);
 - 两者都会改
 - 两者都不会改
- 递归函数必须具备：
 - 全局变量
 - 终止条件
 - 指针参数
 - static 关键字
- 局部 static int n = 0; 的特点是：
 - 每次进入函数都重新初始化为 0
 - 仅第一次初始化，之后保持上次的值
 - 作用域是整个程序所有文件
 - 不能在函数内使用

5. 下列说法错误的是：
- A. 函数可以返回 `void`
 - B. 可以返回指针，但不应返回局部变量地址
 - C. 数组作参数时实际传递的是首元素地址
 - D. 值传递可以直接修改调用者的实参变量本身
6. `int max(int a, int b);` 是：
- A. 函数定义
 - B. 函数声明（原型）
 - C. 函数调用
 - D. 预处理指令
7. `static` 修饰全局函数/全局变量时，主要作用是：
- A. 使其生命周期变短
 - B. 限制为仅本文件（翻译单元）可见
 - C. 强制放在寄存器
 - D. 变成宏
8. 关于 `main`，正确的是：
- A. 可以没有，程序仍可移植启动
 - B. 通常返回 `int`，`return 0` 表示成功
 - C. 必须命名为 `Main`
 - D. 不能接收命令行参数
9. 函数定义中，无返回值时应写返回类型为：
- A. `int`（强制）
 - B. `void`
 - C. `null`
 - D. `empty`
10. 下列关于递归的说法，错误的是：
- A. 必须有终止条件
 - B. 每次调用通常有独立的栈帧
 - C. 递归一定比循环更快
 - D. 可将大问题化为同类小问题
11. `extern` 的常见用途是：
- A. 定义宏
 - B. 声明其他文件中定义的变量或函数
 - C. 分配堆内存
 - D. 结束程序
12. 数组作为函数参数时，函数内修改 `a[i]`：
- A. 绝不影响调用者数组
 - B. 通常会影响调用者数组（因传的是地址）
 - C. 只能读不能写
 - D. 会复制整个数组后再改副本

填空题

13. C 语言参数默认是 `__` 传递；若要通过函数修改实参，通常传递 `__`。
14. 函数 `__` 给出返回类型、函数名与参数类型；函数 `__` 包含具体语句实现。
15. `static` 局部变量的生命周期是 `__`，作用域仍在 `__` 内。
16. `int main(void)` 中 `return 0;` 通常表示 `__`。
17. 带命令行参数的 `main` 常见原型是 `int main(int argc, __)`。
18. 递归求阶乘时，终止条件通常写成当 `n <= 1` 时返回 `__`。

19. 头文件中通常放函数 __, 源文件中放函数 __。

读程序写结果

20. 写出程序输出:

```
#include <stdio.h>
void f(int x) {
    x = x + 1;
}
int main(void) {
    int a = 3;
    f(a);
    printf("%d\n", a);
    return 0;
}
```

21. 写出程序输出:

```
#include <stdio.h>
void f(int *p) {
    *p = *p + 1;
}
int main(void) {
    int a = 3;
    f(&a);
    printf("%d\n", a);
    return 0;
}
```

22. 写出程序输出:

```
#include <stdio.h>
int fact(int n) {
    if (n <= 1) return 1;
    return n * fact(n - 1);
}
int main(void) {
    printf("%d\n", fact(4));
    return 0;
}
```

23. 写出程序输出:

```
#include <stdio.h>
int counter(void) {
    static int n = 0;
    n++;
    return n;
}
int main(void) {
    printf("%d %d %d\n", counter(), counter(), counter());
    return 0;
}
```

24. 写出程序输出:

```
#include <stdio.h>
int sum(int a[], int n) {
    int s = 0;
    for (int i = 0; i < n; i++)
        s += a[i];
    return s;
}
int main(void) {
    int arr[] = {1, 2, 3, 4};
    printf("%d\n", sum(arr, 4));
    return 0;
}
```

25. 写出程序输出:

```
#include <stdio.h>
int g = 1;
void inc(void) {
    g++;
}
int main(void) {
    inc();
    inc();
    printf("%d\n", g);
    return 0;
}
```

简答或编程

26. 编写函数 `int add(int a, int b)`，返回两数之和，并在 `main` 中调用输出 `add(2, 3)`。
27. 编写函数 `void swap(int *a, int *b)`，交换两个整型变量的值。
28. 编写递归函数 `int fib(int n)`，返回斐波那契数列第 n 项（约定 `fib(1)=1, fib(2)=1`）。
29. 简述声明放在头文件、定义放在源文件的好处。
30. 说明为什么不能返回局部变量的地址；若需要“返回”多个值可以怎么做？

二、参考答案与解析

1. 答案: C

解析: 声明是原型, 定义是带函数体的实现。

2. 答案: B

解析: 值传递改的是副本; 通过指针解引用可改实参对象。

3. 答案: B

解析: 没有终止条件会无限递归直至栈溢出。

4. 答案: B

解析: `static` 局部变量只初始化一次, 值在调用间保留。

5. 答案: D

解析: 值传递不能直接改实参本身。

6. 答案: B

解析: 以分号结束、无函数体, 是声明。

7. 答案: B

解析: 内部链接, 避免与其他文件同名冲突。

8. 答案: B

解析: 标准入口返回 `int`; 也可用 `argc/argv`。

9. 答案: B

解析: `void` 表示无返回值。

10. 答案: C

解析: 递归未必更快, 还可能有额外开销。

11. 答案: B

解析: 告诉编译器标识符在别处定义。

12. 答案: B

解析: 数组传参退化成指针, 改的是原数组元素。

13. 答案: 值; 地址 (指针)

解析: 这也是 `swap` 必须用指针的原因。

14. 答案: 声明; 定义

解析: 先声明后调用, 或保证定义出现在调用之前。

15. 答案: 整个程序运行期间; 定义它的函数 (块)

解析: 寿长域不长。

16. 答案: 程序成功 (正常) 结束

解析: 非 0 常表示错误码。

17. 答案: `char *argv[]` (或 ``char argv``) **

解析: `argc` 参数个数, `argv` 参数字符串数组。

18. 答案: 1

解析: `0! = 1`, `1! = 1`。

19. 答案: 声明 (原型) ; 定义 (实现)

解析: 多文件工程基本约定。

20. 答案:

3

解析: 形参 ``x`` 是副本, 实参 ``a`` 不变。

21. 答案:

4

解析: 通过指针修改了 ``a``。

22. 答案:

24

解析: ``4*3*2*1=24``。

23. 答案:

1 2 3

解析: ``static n`` 跨调用累加。(实参求值顺序虽实现相关, 三个调用依次发生时通常输出 1 2 3。)

24. 答案:

10

解析: `1+2+3+4=10`。

25. 答案:

3

解析: 两次 ``inc`` 后全局 ``g`` 从 1 变为 3。

26. 参考代码：

```
#include <stdio.h>
int add(int a, int b) { return a + b; }
int main(void) {
    printf("%d\n", add(2, 3)); /* 5 */
    return 0;
}
```

27. 参考代码：

```
void swap(int *a, int *b) {
    int t = *a;
    *a = *b;
    *b = t;
}
```

28. 参考代码：

```
int fib(int n) {
    if (n <= 2) return 1;
    return fib(n - 1) + fib(n - 2);
}
```

29. 答案要点：

接口与实现分离，便于多文件共享声明、隐藏实现细节、减少重复与链接冲突，提高可维护性。

30. 答案要点：

局部变量在函数返回后生命周期结束，地址悬空。可通过指针/引用参数写回、返回结构体、或返回动态分配内存（需约定谁释放）。

三、本章知识点总结

主题	口诀
声明/定义	声明说有，定义写出
传参	默认传值；改实参传址
递归	要有出口
static 局部	寿长域不长
返回指针	勿返局部地址

第 6 章 · 指针

本章知识点速览

- `&` 取地址, `*` 解引用; 指针保存的是地址
- `p+1` 按元素大小移动, 不是固定 1 字节
- 数组名多数情况下等价于首元素地址; `a[i]` 等价 `*(a+i)`
- `NULL` 表示空指针; 解引用未初始化/悬空指针是未定义行为
- `const int *p` 不能改 `*p`; `int *const p` 不能改 `p` 的指向

一、题目

选择题

1. 若 `int x = 5; int *p = &x;`, 则 `*p` 的值是:
 - A. `x` 的地址
 - B. 5
 - C. `p` 的地址
 - D. 不确定
2. 关于 `NULL`, 正确的是:
 - A. 指向地址 1
 - B. 表示空指针, 不指向有效对象
 - C. 与未初始化指针完全一样安全
 - D. 只能用于字符指针
3. `int a[5]; int *p = a;` 则 `p + 1` 指向:
 - A. 下一字节
 - B. `a[1]`
 - C. `a[0]`
 - D. 数组末尾之后固定位置
4. `const int *p` 的含义是:
 - A. 指针 `p` 不能改变指向
 - B. 不能通过 `p` 修改所指向的 `int`
 - C. `p` 与 `*p` 都不能改
 - D. `p` 只能指向字符串
5. 下列属于危险用法的是:
 - A. `int *p = NULL;`
 - B. `int x; int *p = &x; *p = 1;`
 - C. 返回局部变量的地址并继续解引用
 - D. `p = &x;` 后再读 `*p`
6. `a[i]` 等价于:
 - A. `a + i`
 - B. `*(a + i)`
 - C. `&a[i] + 1`
 - D. `*a + i` (对任意 `i` 都正确)
7. 常见 64 位环境下, 一般 `sizeof(int *)` 是:
 - A. 4
 - B. 8
 - C. 与 `sizeof(int)` 一定相同
 - D. 1
8. `void *` 指针的特点是:
 - A. 可直接解引用任意类型而不转换

- B. 通用指针，使用前通常需转换为具体类型指针
 - C. 只能指向 `void` 函数
 - D. 不能赋值为 `NULL`
9. 若 `int *p = NULL;`，则执行 `*p = 1;`：
- A. 一定安全地把地址 0 写成 1
 - B. 属于危险/未定义行为
 - C. 编译器一定拒绝
 - D. 等价于什么也不做
10. `int **pp;` 表示：
- A. 指向 `int` 的指针
 - B. 指向“指向 `int` 的指针”的指针
 - C. 二维数组
 - D. 函数指针
11. 关于指针比较，正确的是：
- A. 任意两个无关指针相减都有明确标准结果
 - B. 同数组内两指针相减可得元素个数差
 - C. 指针不能用 `==` 比较
 - D. `p1 > p2` 在任何情况下都可移植比较
12. `int (*fp)(int);` 声明的是：
- A. 返回指针的函数
 - B. 指向函数的指针，该函数接受 `int` 并返回 `int`
 - C. 整型数组
 - D. 指向 `int` 的指针数组

填空题

13. 取变量地址用运算符 `&`；通过指针访问对象用 `*`。
14. 常见 64 位环境下，一般指针 `sizeof(p)` 为 `8` 字节。
15. `int *const p` 表示 `p` 不可改，但通常可以通过 `*p` `修改`（若指向非常量）。
16. 指针作为函数参数时，可在函数内通过 `*p` 修改调用者的变量。
17. 释放动态内存后，最好把指针置为 `NULL`，避免悬空使用。
18. `p[i]` 等价于 `*(p+i)`。
19. `const int *const p` 表示既不能改 `p`，也不能改 `*p`。

读程序写结果

20. 写出程序输出：

```
#include <stdio.h>
int main(void) {
    int x = 10;
    int *p = &x;
    *p = 20;
    printf("%d %d\n", x, *p);
    return 0;
}
```

21. 写出程序输出：

```
#include <stdio.h>
int main(void) {
    int a[3] = {1, 2, 3};
    int *p = a;
    printf("%d %d %d\n", *p, *(p + 1), p[2]);
    return 0;
}
```

22. 写出程序输出 (常见 64 位, `int` 4 字节) :

```
#include <stdio.h>
int main(void) {
    int a[5];
    int *p = a;
    printf("%zu\n", sizeof(a));
    printf("%zu\n", sizeof(p));
    return 0;
}
```

23. 写出程序输出:

```
#include <stdio.h>
void add1(int *p) {
    (*p)++;
}
int main(void) {
    int x = 7;
    add1(&x);
    printf("%d\n", x);
    return 0;
}
```

24. 写出程序输出:

```
#include <stdio.h>
int main(void) {
    int a[4] = {10, 20, 30, 40};
    int *p = a + 1;
    printf("%d %d\n", *p, *(p + 2));
    return 0;
}
```

25. 写出程序输出:

```
#include <stdio.h>
int main(void) {
    int x = 5, y = 8;
    int *p = &x;
    int **pp = &p;
    **pp = y;
    printf("%d %d\n", x, *p);
    return 0;
}
```

简答或编程

26. 说明 `const int *p`、`int *const p`、`const int *const p` 三者区别。
27. 列举至少三种指针常见错误（如未初始化解引用、悬空、越界等），并各用一句话说明危害。
28. 编写函数 `void set_to_zero(int *p)`，若 `p` 非 `NULL` 则把所指对象置 0。
29. 解释为何数组传入函数后，函数内 `sizeof(形参)` 往往得不到原数组长度。
30. 编写函数 `int *find_max(int *a, int n)`，返回数组中最大元素的指针（`n>0`，假设非空）。

二、参考答案与解析

1. 答案：B

解析：`*p` 解引用得到 `x` 的值 5。

2. 答案：B

解析：`NULL` 是空指针约定值；仍不可解引用。

3. 答案：B

解析：指针加法按所指数组元素大小步进。

4. 答案：B

解析：指向“常量”的指针，限制的是 `*p`。

5. 答案：C

解析：局部变量生命周期结束，返回其地址形成悬空指针。

6. 答案：B

解析：下标运算定义为 `*(a+i)`。

7. 答案：B

解析：常见 64 位下指针宽度为 8 字节。

8. 答案：B

解析：`void *` 不能直接解引用，需转换。

9. 答案：B

解析：解引用空指针是未定义行为。

10. 答案：B

解析：二级指针。

11. 答案：B

解析：指针减法要求指向同一数组（或一端之后）。

12. 答案：B

解析：括号表明 `fp` 是指针，后面是函数类型。

13. 答案：`&`；`*`

解析：取址与解引用成对理解。

14. 答案：8

解析：常见 64 位 LP64/LLP64 下指针 8 字节。

15. 答案：指针本身（指向）；修改目标对象

解析：`*const` 锁的是指针变量。

16. 答案：解引用（`*p = ...`）

解析：传址 + 解引用实现“输出参数”。

17. 答案：`NULL`

解析：避免野指针/悬空指针继续使用。

18. 答案：`*(p + i)`

解析：与数组下标规则一致。

19. 答案：`p` 的指向；`*p` 的值

解析：双向锁定。

20. 答案：

20 20

解析：通过指针修改了 ``x``。

21. 答案：

1 2 3

解析：``p`` 指向首元素，``p[i]`` 与 ``*(p+i)`` 等价。

22. 答案：

20

8

解析：数组 ``sizeof`` 为 `5*4=20`；指针 ``sizeof`` 为 8。

23. 答案:

8

解析: 函数通过指针把 `x` 加 1。

24. 答案:

20 40

解析: `p` 指向 `a[1]`; `p+2` 指向 `a[3]`。

25. 答案:

8 8

解析: 通过 `\*\*pp` 把 `x` 改成 `y` 的值 8。

26. 答案要点:

`const int *p`: 不能改 `*p`, 可改 `p` 指向; `int *const p`: 不能改 `p`, 可改 `*p`; `const int *const p`: 两者都不能改。

27. 答案要点:

未初始化解引用→未定义/崩溃; 返回局部地址→悬空; 越界访问→破坏数据; 释放后使用/重复 free→堆损坏。

28. 参考代码:

```
void set_to_zero(int *p) {
    if (p != NULL)
        *p = 0;
}
```

29. 答案要点:

数组传参退化为指针, 形参是指针副本, `sizeof` 得到的是指针大小而非原数组字节数, 故需额外传长度。

30. 参考代码:

```
int *find_max(int *a, int n) {
    int *maxp = a;
    for (int i = 1; i < n; i++)
        if (a[i] > *maxp)
            maxp = &a[i];
    return maxp;
}
```

三、本章知识点总结

写法	含义
<code>&x</code> / <code>*p</code>	取址 / 解引用
<code>p+1</code>	下一元素
<code>a[i]</code>	<code>*(a+i)</code>
<code>NULL</code>	空指针，勿解引用
<code>const</code> 位置	左定值、右定针

第 7 章 · 数组

本章知识点速览

- 下标从 0 到 n-1；C 不检查越界
- 部分初始化其余为 0；`int a[]={1,2,3}` 长度由初值决定
- `sizeof(a)/sizeof(a[0])` 求一维数组长度（仅数组未被退化时）
- 数组作函数参数会退化为指针，必须同时传长度
- 常见操作：遍历、求和、求最大、逆序；二维数组按行优先存储

一、题目

选择题

- `int a[5] = {1, 2};` 后，`a[3]` 的值是：
A. 1
B. 2
C. 0
D. 不确定
- 数组下标合法范围是（长度为 n）：
A. 1 ~ n
B. 0 ~ n
C. 0 ~ n-1
D. 1 ~ n-1
- 关于数组越界，正确的是：
A. 编译器一定报错并拒绝运行
B. C 语言运行时通常不检查，属未定义行为
C. 越界会自动扩展数组
D. 越界总是得到 0
- 设 `int a[10];`（`int` 为 4 字节），`sizeof(a)` 通常为：
A. 10
B. 4

- C. 40
D. 8
5. 数组作为函数参数时：
A. 函数内可用 `sizeof` 可靠得到元素个数
B. 传递的是整个数组的拷贝
C. 退化为指针，通常需额外传入长度
D. 不能传递数组
6. 二维数组 `int a[2][3] = {{1,2,3},{4,5,6}};` 中 `a[1][2]` 是：
A. 1
B. 3
C. 5
D. 6
7. `int a[] = {1, 2, 3, 4, 5};` 的长度是：
A. 4
B. 5
C. 6
D. 不确定
8. 关于数组名，错误的是：
A. 多数情况下可当作首元素地址使用
B. `a[i]` 与 `*(a+i)` 等价
C. 数组名是可以 `a++` 自增的指针变量
D. 对数组做 `sizeof` 通常得到整块字节数
9. `int a[2][3];` 的元素总个数是：
A. 2
B. 3
C. 5
D. 6
10. 下列初始化能把 `int a[5]` 全部置 0 的是：
A. `int a[5];` (局部)
B. `int a[5] = {0};`
C. `int a[5] = {1};` (其余不定)
D. 只能循环赋值，没有别的办法
11. 向函数传递二维数组时，下列声明较合理的是：
A. `void f(int a[][ ], int rows)`
B. `void f(int a[][4], int rows)`
C. `void f(int a[ ][ ], int rows)` (两维都可省略)
D. `void f(int a)`
12. 若 `int a[5] = {1,2,3,4,5};`，则 `&a[0]` 与 `a`：
A. 值通常相同 (都指向首元素)
B. 类型与含义完全不可能相关
C. 一定不相等
D. `&a[0]` 是非法写法

填空题

13. `int a[] = {10, 20, 30, 40};` 则数组长度为 ____。
14. 求一维数组元素个数常用表达式： ____。
15. 二维数组在内存中通常按 ____ 优先存储。
16. 向函数传递二维数组时，声明中一般需要指定 ____ 数。

17. 数组下标从 `_` 开始；最后一个元素下标是 `_`。
18. 常见操作中，首尾交换向中间靠拢可实现数组 `_`。
19. 一维数组 `int a[n];` 在常见环境下占 `_` × `sizeof(int)` 字节（忽略对齐问题）。

读程序写结果

20. 写出程序输出：

```
#include <stdio.h>
int main(void) {
    int a[5] = {1, 2};
    printf("%d %d %d\n", a[0], a[1], a[4]);
    return 0;
}
```

21. 写出程序输出（常见 64 位）：

```
#include <stdio.h>
int main(void) {
    int a[4] = {1, 2, 3, 4};
    printf("%zu\n", sizeof(a) / sizeof(a[0]));
    return 0;
}
```

22. 写出程序输出：

```
#include <stdio.h>
int main(void) {
    int a[2][3] = {{1, 2, 3}, {4, 5, 6}};
    printf("%d %d\n", a[0][2], a[1][0]);
    return 0;
}
```

23. 写出程序输出：

```
#include <stdio.h>
void f(int a[], int n) {
    printf("%d\n", a[0] + a[n - 1]);
}
int main(void) {
    int arr[] = {2, 4, 6, 8};
    f(arr, 4);
    return 0;
}
```

24. 写出程序输出：

```
#include <stdio.h>
int main(void) {
    int a[5] = {1, 2, 3, 4, 5};
    int s = 0;
    for (int i = 0; i < 5; i++)
        s += a[i];
    printf("%d\n", s);
    return 0;
}
```

25. 写出程序输出:

```
#include <stdio.h>
int main(void) {
    int a[3][3] = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
    printf("%d %d\n", a[2][1], (*(a + 1) + 2));
    return 0;
}
```

简答或编程

26. 编写程序: 对数组 `int a[5] = {3, 9, 2, 7, 5};` 找出最大值并输出。
27. 编写函数 `void reverse(int a[], int n)`, 将数组元素原地逆序。
28. 说明为什么 `int a[10] = {0};` 能把整个数组置 0。
29. 简述数组名与指针的联系及差别 (至少两点)。
30. 编写函数 `int sum_array(const int a[], int n)`, 返回数组元素之和。

二、参考答案与解析

1. 答案: C

解析: 未显式初始化的剩余元素补 0。

2. 答案: C

解析: 下标从 0 开始到 n-1。

3. 答案: B

解析: C 不做边界检查, 越界是未定义行为。

4. 答案: C

解析: $10 \times 4 = 40$ 字节。

5. 答案: C

解析: 传参退化成指针, 长度要另传。

6. 答案: D

解析: 第二行第三列是 6。

7. 答案: B

解析：初始化列表 5 个元素，长度即为 5。

8. 答案：C

解析：数组名不是可修改的左值指针变量，不能 `a++`。

9. 答案：D

解析： $2 \times 3 = 6$ 。

10. 答案：B

解析：提供初始化列表时其余元素补 0。

11. 答案：B

解析：列数必须已知，以便计算行地址。

12. 答案：A

解析：二者都指向首元素；类型上 `&a[0]` 为 `int *`，`&a` 才是数组指针。

13. 答案：4

解析：省略长度时由初始化列表决定。

14. 答案：`sizeof(a)/sizeof(a[0])`

解析：仅当 `a` 仍是数组对象时成立。

15. 答案：行

解析：行优先：先存完一行再下一行。

16. 答案：列

解析：如 `void f(int a[][4], int rows)`。

17. 答案：0; n-1

解析：长度 n 的数组下标范围。

18. 答案：逆序

解析：经典双指针交换。

19. 答案：n

解析：连续存放 n 个 `int`。

20. 答案：

1 2 0

解析：前两元素为 1、2，其余为 0。

21. 答案：

4

解析：4 个 `int` 元素。

22. 答案：

3 4

解析：`a[0][2]=3`，`a[1][0]=4`。

23. 答案：

10

解析：`a[0]+a[3] = 2+8 = 10`。

24. 答案：

15

解析：1+2+3+4+5=15。

25. 答案：

8 6

解析：`a[2][1]=8`；`\*(\*(a+1)+2)` 即 `a[1][2]=6`。

26. 参考代码：

```
#include <stdio.h>
int main(void) {
    int a[5] = {3, 9, 2, 7, 5};
    int max = a[0];
    for (int i = 1; i < 5; i++)
        if (a[i] > max) max = a[i];
    printf("%d\n", max); /* 9 */
    return 0;
}
```

27. 参考代码：

```
void reverse(int a[], int n) {
    for (int i = 0, j = n - 1; i < j; i++, j--) {
        int t = a[i];
        a[i] = a[j];
        a[j] = t;
    }
}
```

28. 答案要点:

提供初始化列表时，未列出的元素自动置 0；只写 `{0}` 即首元素为 0、其余亦为 0。

29. 答案要点:

联系：`a[i]` 与 `*(a+i)` 等价，数组名常可当作首元素指针使用。差别：数组名不是可修改的指针变量；对数组做 `sizeof` 得整块大小，对指针得指针大小。

30. 参考代码:

```
int sum_array(const int a[], int n) {
    int s = 0;
    for (int i = 0; i < n; i++)
        s += a[i];
    return s;
}
```

三、本章知识点总结

考点	速记
下标	0 ~ n-1, 越界危险
初始化	剩余补 0; <code>{0}</code> 全零
sizeof	未退化前可算长度
作参数	退化指针, 必传 n
常考操作	最大 / 逆序 / 求和
二维	行优先, 传参指定列

第 8 章 · 字符串

本章知识点速览

- 字符串本质是以 `'\0'` 结尾的字符序列；字面量 `"hi"` 占 3 字节
- `strlen` 不计 `'\0'`；`strcpy` / `strcat` / `strcmp` 需目标缓冲区足够大
- `char s[]` 可改内容；`char *p = "hi"` 指向常量区，不可写
- 输入优先 `fgets`；`scanf("%s")` 遇空白停止且易溢出
- 比较内容用 `strcmp`，不要用 `==`（比较的是地址）

一、题目

选择题

1. C 语言中字符串以什么作为结束标志?
 - A. 空格
 - B. `'\n'`
 - C. `'\0'`
 - D. EOF
2. 执行 `char s[] = "ABC";` 后, `sizeof(s)` 在常见环境下等于?
 - A. 3
 - B. 4
 - C. 8
 - D. 不定
3. 下列关于 `strlen("hello")` 的说法正确的是?
 - A. 返回 6
 - B. 返回 5
 - C. 返回 5 或 6 均可
 - D. 编译错误
4. `char *p = "hi"; p[0] = 'H';` 的结果通常是?
 - A. 合法, 字符串变为 `"Hi"`
 - B. 未定义行为 / 可能崩溃 (改写字符串常量)
 - C. 编译一定失败
 - D. `p` 变为空指针
5. 比较两个字符串内容是否相等, 应使用?
 - A. `s1 == s2`
 - B. `strcmp(s1, s2) == 0`
 - C. `s1 = s2`
 - D. `strlen(s1) == strlen(s2)`
6. 关于 `scanf("%s", buf)`, 下列哪项正确?
 - A. 可以读入带空格的一行
 - B. 遇空白停止, 且不检查长度时可能溢出

C. 会自动在末尾加 `'\n'`

D. 等价于 `fgets`

7. `strcat(dst, src)` 的作用是?

A. 用 `src` 覆盖 `dst`

B. 将 `src` 接到 `dst` 末尾 (覆盖 `dst` 原有的 `'\0'`)

C. 交换 `dst` 与 `src`

D. 比较 `dst` 与 `src`

8. 下列关于 `gets` 的说法正确的是?

A. 推荐使用, 比 `fgets` 更安全

B. 已废弃且无法限制长度, 禁止使用

C. 会自动截断超长输入

D. 声明在 `string.h`

9. `strchr("hello", 'l')` 返回的是?

A. 字符 `'l'` 的 ASCII 值

B. 指向第一次出现 `'l'` 的指针

C. 子串 `"llo"` 的副本

D. 下标 2

10. `strncpy(dst, src, n)` 若 `src` 长度大于等于 `n`, 常见问题是?

A. 一定自动补 `'\0'`

B. 可能不会写入结束符, 需自行保证 `'\0'`

C. 编译错误

D. 等价于 `strcpy`

11. `char s[5] = "abcd";` 之后, `s[4]` 的值是?

A. `'d'`

B. `'\0'`

C. 未定义

D. `' '`

填空题

12. `"Hello"` 在内存中共占 `__` 字节 (含结束符)。

13. `strcmp("ab", "ac")` 的返回值 `__` 0 (填 `<`、`>` 或 `==`)。

14. 安全地读一行输入到数组 `char s[100]`，常用 `_____ (s, sizeof(s), stdin);`。
15. `char a[] = "hi"; char *p = "hi";` 中，可修改字符内容的是 `__`（填 `a` 或 `p`）。
16. `scanf("%99s", buf)` 中的 `99` 作用是限制最多读入 `__` 个字符（不含结束符）。
17. `puts(s)` 输出字符串后还会额外输出一个 `__`。
18. 去掉 `fgets` 读入行末换行，可用 `s[strcspn(s, "\n")] = _____;`。

读程序写结果

19. 写出输出：

```
#include <stdio.h>
#include <string.h>
int main(void) {
    char s[] = "Hello";
    printf("%zu\n", strlen(s));
    printf("%zu\n", sizeof(s));
    return 0;
}
```

20. 写出输出：

```
#include <stdio.h>
#include <string.h>
int main(void) {
    char a[20] = "ab";
    char b[] = "cd";
    strcat(a, b);
    printf("%s %zu\n", a, strlen(a));
    return 0;
}
```

21. 写出输出：

```
#include <stdio.h>
#include <string.h>
int main(void) {
    char s1[] = "apple";
    char s2[] = "apply";
    printf("%d\n", strcmp(s1, s2) < 0);
    printf("%d\n", s1 == s2);
    return 0;
}
```

22. 写出输出：


```
#include <stdio.h>
#include <string.h>
int main(void) {
    char buf[10] = "xyz";
    strcpy(buf, "hi");
    printf("%s %d\n", buf, (int)strlen(buf));
    return 0;
}
```

23. 写出输出：

```
#include <stdio.h>
#include <string.h>
int main(void) {
    char s[] = "C-Language";
    char *p = strchr(s, '-');
    printf("%s\n", p);
    printf("%c\n", *(p + 1));
    return 0;
}
```

24. 写出输出：

```
#include <stdio.h>
#include <string.h>
int main(void) {
    char s[20] = "12";
    strcat(s, "34");
    printf("%zu %s\n", strlen(s), s);
    printf("%d\n", strcmp(s, "1234") == 0);
    return 0;
}
```

简答或编程

25. 简述：为什么不能用 `==` 判断两个字符数组内容是否相同？
26. 编写函数 `int my_strlen(const char *s);`，不用库函数求字符串长度。
27. 编写程序：读入一个不含空格的字符串，输出其反转结果（可用数组，注意 `'\0'`）。
28. 编写函数 `void my_strcpy(char *dst, const char *src);`，实现字符串复制（含结束符）。
29. 编写程序：读入一行字符串（可含空格），统计其中数字字符个数（用 `isdigit` 或自行判断）。
30. 简述 `strcpy` 与 `strncpy` 的主要区别及使用时应注意什么。

二、参考答案与解析

1. 答案：C
解析：字符串以空字符 `'\0'`（值为 0）结尾。
2. 答案：B
解析：`'A', 'B', 'C', '\0'` 共 4 字节。
3. 答案：B
解析：`strlen` 不计结束符，`"hello"` 长度为 5。

4. 答案: B

解析: 字符串字面量通常在只读区, 写操作是未定义行为。

5. 答案: B

解析: `strcmp` 比较内容; `==` 比较指针地址。

6. 答案: B

解析: `%s` 遇空白停止; 应限制宽度或用 `fgets`。

7. 答案: B

解析: 从 `dst` 的 `'\0'` 处开始追加 `src` (含新的 `'\0'`)。

8. 答案: B

解析: `gets` 无法限制缓冲区长度, 存在严重溢出风险, 已废弃。

9. 答案: B

解析: `strchr` 返回指向首次出现该字符的指针; 未找到返回 `NULL`。

10. 答案: B

解析: 若复制满 `n` 个字符且未遇到源串结束, `strncpy` 不一定写入 `'\0'`。

11. 答案: B

解析: 字面量末尾的 `'\0'` 写入 `s[4]`。

12. 答案: 6

解析: `Hello` `\0`。

13. 答案: <

解析: 第二字符 `'b' < 'c'`, 故返回负值。

14. 答案: fgets

解析: `fgets` 可限制长度并读入一行。

15. 答案: a

解析: 数组可写; `p` 指向常量不宜修改。

16. 答案: 99

解析: 宽度字段限制最多读入的字符数, 并预留结束符位置。

17. 答案: 换行符 (`'\n'`)

解析: `puts` 输出字符串后再输出换行。

18. 答案: `'\0'`

解析: 把换行位置改成结束符即可去掉行末 `\n`。

19. 答案:

```
5
6
```

解析: `strlen` 不计 `'\0'` 为 5; `sizeof` 含 `'\0'` 为 6。

20. 答案: `abcd 4`

解析: 连接后长度为 4。

21. 答案:

```
1
0
```

解析：字典序 `apple < apply`；两数组地址不同，`==` 为假。

22. 答案： `hi 2`

解析： `strcpy` 覆盖原内容并写上新的 `'\0'`。

23. 答案：

```
-Language
L
```

解析： `p` 指向 `'-'`； `p+1` 为 `'L'`。

24. 答案：

```
4 1234
1
```

解析：连接得 `"1234"`；与 `"1234"` 比较相等。

25. 答案要点： `==` 比较的是数组（退化后的指针）地址，不是逐字符内容；应使用 `strcmp`。

26. 参考代码：

```
int my_strlen(const char *s) {
    int n = 0;
    while (s[n] != '\0') n++;
    return n;
}
```

27. 参考思路：用 `scanf("%99s", s)` 或 `fgets` 读入；首尾交换至中间； `printf("%s\n", s)`。

28. 参考代码：

```
void my_strcpy(char *dst, const char *src) {
    while ((*dst++ = *src++))
        ;
}
```

29. 参考要点： `fgets(s, sizeof(s), stdin)`；遍历； `isdigit((unsigned char)s[i])` 计数并输出。

30. 答案要点： `strcpy` 复制整串含 `'\0'`，易溢出； `strncpy` 最多复制 `n` 字节，可能无结束符。都须保证目标缓冲区足够，必要时手动补 `'\0'`。

三、本章知识点总结

点	记法
结束符	必须有 <code>'\0'</code> ，否则库函数会越界读
strlen / sizeof	长度不含 <code>\0</code> ；数组 <code>sizeof</code> 含 <code>\0</code>
数组 vs 指针	<code>s[]</code> 可改；指向字面量的指针不可改
比较	<code>strcmp</code> ；勿用 <code>==</code>
输入	优先 <code>fgets</code> ； <code>scanf %s</code> 限宽；禁用 <code>gets</code>

第 9 章 · 结构体与枚举

本章知识点速览

- `struct` 把不同类型成员组合在一起；成员用 `.`，指针用 `->`
- `typedef` 可为结构体起别名，省略反复写 `struct`
- `union` 成员共享同一块内存，大小约为最大成员
- `enum` 定义命名整型常量，默认从 0 递增
- 链表结点：数据域 + 指向下一结点的指针

一、题目

选择题

- 访问结构体变量 `s` 的成员 `id`，正确写法是？
 - `s->id`
 - `s.id`
 - `*s.id`
 - `s[id]`
- 若 `struct Student *p;` 已指向有效对象，访问 `score` 应写？
 - `p.score`
 - `p->score`
 - `Student.score`
 - `*p->score` (若 `score` 非指针)
- 关于 `union`，正确的是？
 - 各成员同时独立占有内存
 - 所有成员共享同一段内存

C. 大小等于各成员之和

D. 不能包含 `int`

4. `enum Color { RED, GREEN, BLUE };` 中 `GREEN` 的默认值是?

A. 0

B. 1

C. 2

D. 未定义

5. `typedef struct { int x, y; } Point;` 之后, 下列合法的是?

A. `Point p;`

B. 必须写 `struct Point p;`

C. 不能定义变量

D. `Point` 是宏

6. 下列哪项最适合描述链表结点?

A. 仅含一个 `int`

B. 数据域 + `next` 指针

C. 只能用数组下标链接

D. 必须用 `union`

7. 关于结构体对齐, 下列说法较合理的是?

A. `sizeof(结构体)` 一定等于各成员 `sizeof` 之和

B. 因对齐, `sizeof` 可能大于成员之和

C. 结构体不能含数组成员

D. 对齐只在 32 位系统存在

8. 结构体作函数参数时, 传指针相对传值的主要优点是?

A. 语法更简单

B. 避免大对象拷贝, 且可修改原对象

C. 不需要头文件

D. 自动释放内存

9. `union U { int i; char c[4]; };` 在常见环境下, `sizeof(union U)` 约为?

A. 1

B. 4

C. 8

D. 16

10. 下列关于嵌套结构体的访问，正确的是？

A. 不能嵌套

B. 可用 `p.birthday.year` 这类多级 `.`

C. 只能用指针访问内层

D. 内层必须是 `union`

11. C99 指定初始化 `struct S s = { .id = 1, .score = 90 };` 的含义是？

A. 非法语法

B. 按成员名初始化指定成员

C. 只初始化第一个成员

D. 等价于全部置 0 且不能再赋值

填空题

12. `(*p).id` 与 `_____` 等价 (p 为结构体指针)。

13. `struct` 成员 `_____` 共存；`union` 成员 `_____` 共存 (填“同时/不同时”类表述)。

14. `enum { A = 2, B, C };` 中 `C` 的值为 `_____`。

15. 结构体作函数参数时，大对象更宜传 `_____` (值 / 指针)。

16. 位域 `unsigned int flag : 1;` 中的 `1` 表示该成员占 `_____` 位。

17. 声明结构体数组可用：`struct Student st[3];`，访问第一个元素的 `id` 写作 `_____`。

18. `typedef struct Node Node;` 后，可用 `Node *p;` 代替写 `_____`。

读程序写结果

19. 写出输出：

```
#include <stdio.h>
struct S { int a; char b; };
int main(void) {
    struct S x = {10, 'A'};
    printf("%d %c\n", x.a, x.b);
    return 0;
}
```

20. 写出输出：

```
#include <stdio.h>
enum { OFF, ON = 5, ERR };
int main(void) {
    printf("%d %d %d\n", OFF, ON, ERR);
    return 0;
}
```

21. 写出输出（常见 64 位，忽略对齐差异时按逻辑值）：

```
#include <stdio.h>
union U { int i; char c[4]; };
int main(void) {
    union U u;
    u.i = 0;
    u.c[0] = 1;
    printf("%d\n", u.i != 0);
    return 0;
}
```

22. 写出输出：

```
#include <stdio.h>
typedef struct { int x, y; } Point;
int main(void) {
    Point a = {3, 4}, *p = &a;
    printf("%d %d\n", a.x + a.y, p->x * p->y);
    return 0;
}
```

23. 写出输出：

```
#include <stdio.h>
typedef struct {
    int year, month, day;
} Date;
typedef struct {
    char name[8];
    Date birthday;
} Person;
int main(void) {
    Person p = {"Amy", {2000, 5, 1}};
    printf("%s %d-%d-%d\n", p.name, p.birthday.year, p.birthday.month, p.birthday.day);
    return 0;
}
```

24. 写出输出：

```
#include <stdio.h>
struct Node { int data; struct Node *next; };
int main(void) {
    struct Node b = {20, NULL};
    struct Node a = {10, &b};
    printf("%d %d\n", a.data, a.next->data);
    return 0;
}
```

简答或编程

25. 简述 `.` 与 `->` 的使用场景差异。
26. 定义结构体 `Node`：含 `int data` 与 `struct Node *next`，并用 `malloc` 创建一个结点赋值为 10、`next` 为 `NULL`（写出关键代码即可）。
27. 用 `typedef` 定义二维点 `Point`（`double x, y`），并写函数原型：计算两点距离 `double dist(Point a, Point b);`。
28. 定义学生结构体（学号、姓名、成绩），编写函数按成绩降序比较的思路（可用于 `qsort` 的比较函数要点）。
29. 简述 `struct` 与 `union` 在内存布局与使用上的核心区别。
30. 编写一小段代码：定义枚举星期（`MON...`），并用 `switch` 打印 `WED` 对应的中文“星期三”。

二、参考答案与解析

1. 答案：B

解析：变量用点运算符。

2. 答案：B

解析：指针访问成员用 `->`。

3. 答案：B

解析：共用体共享存储。

4. 答案：B

解析：从 0 起：`RED=0, GREEN=1, BLUE=2`。

5. 答案：A

解析：`typedef` 后可直接写 `Point`。

6. 答案：B

解析：结点含数据与后继指针。

7. 答案：B

解析：编译器可能插入填充字节以满足对齐。

8. 答案：B

解析：传指针拷贝的是地址，效率高且可改原对象。

9. 答案：B

解析：大小由最大成员决定，此处约为 `sizeof(int)` 或 `sizeof(c)` 即 4。

10. 答案：B

解析：嵌套成员用多级点运算符（或指针与点组合）。

11. 答案：B

解析：C99 支持按成员名指定初始化。

12. 答案： `p->id`

解析：二者等价。

13. 答案：同时；不同时（共享内存，同一时刻有效解释通常一个）

解析：对照 struct / union。

14. 答案：4

解析： `A=2, B=3, C=4`。

15. 答案：指针

解析：避免大块拷贝，且可修改原对象。

16. 答案：1

解析：位域宽度以比特计。

17. 答案： `st[0].id`

解析：数组元素是结构体变量，用 `.`。

18. 答案： `struct Node *p;`

解析： `typedef` 省略反复写 `struct`。

19. 答案： `10 A`

解析：初始化并按成员输出。

20. 答案： `0 5 6`

解析： `OFF` 默认 0； `ON` 指定 5；其后递增为 6。

21. 答案： `1`

解析：改写 `c[0]` 使整型表示非 0（小端下为 1）。

22. 答案： `7 12`

解析： `3+4=7`； `3*4=12`。

23. 答案： `Amy 2000-5-1`

解析：嵌套初始化与访问。

24. 答案： `10 20`

解析： `a.next` 指向 `b`。

25. 答案要点：结构体（或联合）变量用 `.`；指向结构体的指针用 `->`（等价 `(*p).`）。

26. 参考代码：

```
typedef struct Node {
    int data;
    struct Node *next;
} Node;
Node *p = (Node *)malloc(sizeof(Node));
p->data = 10;
p->next = NULL;
```

27. 参考:

```
typedef struct { double x, y; } Point;
double dist(Point a, Point b);
```

28. **参考要点:** 比较函数接收两个 `const void *`, 转为学生指针后比较 `score`; 降序可返回 `pb->score - pa->score` (注意溢出时用条件返回)。
29. **答案要点:** `struct` 各成员独立占位、可同时有效; `union` 共享同一内存, 写一个会覆盖另一个的位模式, 大小约为最大成员。
30. **参考要点:**

```
enum Week { MON, TUE, WED, THU, FRI, SAT, SUN };
enum Week d = WED;
switch (d) {
case WED: printf("星期三\n"); break;
/* ... */
}
```

三、本章知识点总结

- `s.m` / `p->m`: 变量与指针访问
- `typedef struct {...} T;`: 简化类型名
- `union`: 省空间、多视图; 注意覆盖
- `enum`: 命名整数; 可指定起始值
- 链表结点: `data` + `next`, 配合动态分配
- 对齐: `sizeof` 可能大于成员之和; 大结构体宣传指针

第 10 章 · 预处理

本章知识点速览

- `#include <>` 搜系统目录; `#include ""` 先搜当前目录
- `#define` 为文本替换: 带宏参数与整体都要加括号, 并警惕副作用
- 条件编译: `#ifdef` / `#ifndef` / `#if ... #endif`
- 头文件守卫: `#ifndef XXX_H ... #define XXX_H ... #endif`
- 预定义宏: `__FILE__`、`__LINE__`、`__DATE__`、`__TIME__`、`__func__`

一、题目

选择题

1. 预处理指令行末通常?
 - A. 必须加分号
 - B. 不加分号
 - C. 必须加逗号
 - D. 必须加大括号
2. `#include <stdio.h>` 与 `#include "stdio.h"` 的主要区别是?
 - A. 完全相同
 - B. 搜索路径约定不同
 - C. 后者不能包含标准库
 - D. 前者不能包含标准库
3. `#define SQUARE(x) x * x` 展开 `SQUARE(1+2)` 的问题是?
 - A. 无问题, 结果为 9
 - B. 变成 `1+2 * 1+2`, 结果为 5
 - C. 编译错误
 - D. 变成 `(1+2)*(1+2)`
4. 头文件防止重复包含, 常用?
 - A. `#pragma once` 或 include guard
 - B. 只能靠程序员不重复写
 - C. `#undef`
 - D. `typedef`
5. 下列哪项是预定义宏?
 - A. `NULL`
 - B. `__LINE__`
 - C. `EOF`
 - D. `stdin`
6. `#ifdef DEBUG ... #endif` 的含义是?
 - A. 运行时若 `DEBUG` 非 0 则执行
 - B. 若已定义宏 `DEBUG` 则编译该段
 - C. 删除 `DEBUG` 变量
 - D. 链接时包含调试库

7. 关于宏与 `const`，正确的是？
- A. 宏有类型检查
 - B. 宏在预处理阶段文本替换
 - C. `const` 在预处理阶段替换
 - D. 宏一定优于 `const`
8. `#undef MAX` 的作用是？
- A. 定义宏 MAX
 - B. 取消已定义的宏 MAX
 - C. 输出错误
 - D. 包含头文件
9. 下列关于 `#error "msg"` 的说法正确的是？
- A. 运行时报错
 - B. 预处理阶段产生错误并显示信息
 - C. 等价于 `assert`
 - D. 只在链接时有效
10. `#if defined(X)` 与下列哪项最接近？
- A. `#ifdef X`
 - B. `#ifndef X`
 - C. `#undef X`
 - D. `#pragma X`
11. 宏替换发生在编译流程的哪个阶段？
- A. 链接
 - B. 汇编
 - C. 预处理
 - D. 运行时

填空题

12. 推荐写法：`#define MAX(a,b) (____)`（填出使比较安全的括号形式：`(a)>(b)?(a):(b)` 并整体加括号）。
13. `#define SQ(x) ((x)*(x))` 时，`SQ(i++)` 仍可能有 __（副作用被求值两次）。
14. 头文件守卫三件套：`#ifndef`、`#define`、____。
15. 输出当前源文件名可用 `printf("%s\n", ____);`。
16. 取消宏定义使用指令 __。

17. `#ifndef HEADER_H` 表示：若宏 `HEADER_H` 被定义，则编译后续内容。
18. 预定义宏 `_` 给出当前源文件行号。

读程序写结果

19. 写出预处理后逻辑上的输出（设未另定义宏）：

```
#include <stdio.h>
#define PI 3
int main(void) {
    printf("%d\n", PI + 1);
    return 0;
}
```

20. 写出输出：

```
#include <stdio.h>
#define MUL(a,b) ((a)*(b))
int main(void) {
    int x = 3, y = 4;
    printf("%d\n", MUL(x+1, y));
    return 0;
}
```

21. 写出输出：

```
#include <stdio.h>
#define A 1
int main(void) {
    #if A
        printf("yes\n");
    #else
        printf("no\n");
    #endif
    return 0;
}
```

22. 写出输出：

```
#include <stdio.h>
#define INC(x) ((x) + 1)
int main(void) {
    int n = 10;
    printf("%d\n", INC(n) * 2);
    return 0;
}
```

23. 写出输出（注意宏展开把参数写成两份）：

```
#include <stdio.h>
#define TWICE(x) ((x)+(x))
int main(void) {
    int a = 3;
    printf("%d\n", TWICE(a + 1));
    printf("%d\n", a);
    return 0;
}
```

24. 写出输出：

```
#include <stdio.h>
#define VER 2
int main(void) {
    #if VER >= 2
        printf("new\n");
    #elif VER == 1
        printf("old\n");
    #else
        printf("unk\n");
    #endif
    return 0;
}
```

简答或编程

25. 说明为什么带参宏的参数和整个替换体都要加括号，并举一例错误宏。
26. 写出一个名为 `utils.h` 的头文件守卫骨架（宏名自定）。
27. 比较 `#define N 100` 与 `const int N = 100;` 的优缺点（至少各两点）。
28. 简述 `#include` 的本质（文本插入）及重复包含可能带来的问题。
29. 写出一个带参宏 `ABS(x)`，正确加括号，求绝对值。
30. 说明为何不建议把有副作用的表达式（如 `i++`）作为宏实参。

二、参考答案与解析

1. 答案：B
解析：预处理指令不以分号结束。
2. 答案：B
解析：`<>` 与 `""` 搜索顺序约定不同。
3. 答案：B
解析：缺少括号导致运算符优先级错误。
4. 答案：A
解析：`include guard` 或 `#pragma once`。
5. 答案：B
解析：`__LINE__` 等为预定义宏；`NULL` / `EOF` 多为宏但非“预定义宏”考点常指编译器内建那一组。
6. 答案：B
解析：条件编译在预处理阶段决定是否保留代码。

7. 答案: B

解析: 宏无类型, 纯替换; `const` 有类型。

8. 答案: B

解析: `#undef` 取消宏定义。

9. 答案: B

解析: `#error` 在预处理时中止并给出信息。

10. 答案: A

解析: `defined(X)` 判断宏是否已定义, 与 `#ifdef X` 接近。

11. 答案: C

解析: 宏展开属于预处理。

12. 答案: `((a)>(b)?(a):(b))`

解析: 参数与整体加括号。

13. 答案: 副作用 (`i` 自增两次等)

解析: 宏参数可被多次展开。

14. 答案: `#endif`

解析: 关闭条件编译块。

15. 答案: `__FILE__`

解析: 预定义宏给出源文件名字符串。

16. 答案: `#undef`

解析: 取消宏。

17. 答案: 尚未 / 没有

解析: `ifndef` = if not defined。

18. 答案: `__LINE__`

解析: 当前行号。

19. 答案: 4

解析: `PI` 替换为 3。

20. 答案: 16

解析: `((x+1)*(y))` → `4*4=16`。

21. 答案: yes

解析: `A` 为 1, 真分支编译。

22. 答案: 22

解析: 展开为 `((n)+1)*2` → `11*2=22`。

23. 答案:

8
3

解析: 展开为 `((a+1)+(a+1))` → `4+4=8`; `a` 本身未变仍为 3。若把 `i++` 传入此类宏则可能多次求值, 属危险用法。

24. 答案: new

解析: `VER>=2` 成立。

25. **答案要点：**防止优先级错误，如 `#define S(x) x*x` 遇 `S(1+2)`；整体括号防外层运算拆开。另：副作用参数勿用于宏。
26. **参考：**
- ```
#ifndef UTILS_H
#define UTILS_H
/* 声明 */
#endif
```
27. **答案要点：**宏：无类型、全局文本替换、可条件编译；`const`：有类型与作用域、可取地址、调试友好。现代 C 中常量优先 `const` / `enum`，宏更适合条件编译与简单通用替换。
28. **答案要点：**`#include` 把文件内容插入当前位置；重复包含可能导致重复定义错误，故用头文件守卫。
29. **参考：**`#define ABS(x) ((x) >= 0 ? (x) : -(x))`（注意副作用仍可能求值两次）。
30. **答案要点：**宏实参可能被展开多次，副作用（自增、函数调用）会执行多次，结果难预期。

### 三、本章知识点总结

| 主题      | 要点                                                    |
|---------|-------------------------------------------------------|
| include | <code>&lt;&gt;</code> 系统； <code>""</code> 本地优先        |
| 宏       | 加括号；防副作用；无类型                                          |
| 条件编译    | <code>#if/#ifdef/#ifndef</code> + <code>#endif</code> |
| 守卫      | <code>ifndef/define/endif</code>                      |
| 预定义     | <code>__FILE__</code> <code>__LINE__</code> 等         |

## 第 11 章 · 输入输出

### 本章知识点速览

- `printf` / `scanf` 格式符须与类型匹配；`double` 用 `scanf` 时写 `%lf`
- `scanf` 读变量要取地址；数组名读字符串不必再加 `&`
- 输入缓冲区可能残留 `'\n'`，影响后续 `%c` / `getchar`
- `getchar` 返回 `int`，以便表示 `EOF`
- `EOF` 通常为 -1，表示结束或读失败

### 一、题目

#### 选择题

1. 输出整数 `n`，常用格式符是？
- A. `%f`
- B. `%d`



C. `%c`

D. `%s`

2. `double x;` 用 `scanf` 读入, 正确的是?

A. `scanf("%f", &x);`

B. `scanf("%lf", &x);`

C. `scanf("%d", &x);`

D. `scanf("%lf", x);`

3. `char s[20]; scanf("%s", s);` 中为何通常不加 `&`?

A. 语法规则不能加

B. 数组名已表示首元素地址

C. 会自动加

D. `%s` 不需要地址

4. `getchar()` 的返回类型设计为 `int` 的主要原因是?

A. 字符都是 `int`

B. 需要能表示 `EOF` (与普通字符区分)

C. 更快

D. 标准要求返回指针

5. `printf("%.2f", 3.14159);` 输出约为?

A. `3.14`

B. `3.14159`

C. `3`

D. `314`

6. 读完 `scanf("%d", &n);` 后立刻 `scanf("%c", &ch);`, 常见问题是?

A. 一定成功读到下一字符

B. 可能读到残留的换行符

C. `%c` 非法

D. 必须用 `%s`

7. 下列关于 `EOF` 的说法正确的是?

A. 是字符 `'E'`

B. 常定义为 -1, 表示结束/错误

C. 等于 0

D. 只能用于文件不能用于键盘

8. `printf` 输出长整型常用格式符是？

A. `%d`

B. `%ld`

C. `%f`

D. `%c`

9. 关于 `putchar(c)`，正确的是？

A. 输出字符串

B. 输出一个字符

C. 只能输出数字

D. 返回 `void` 且无返回值含义

10. `scanf` 成功读入并赋值的项数作为返回值；若在读第一个数前遇到文件结束，返回值通常是？

A. 0

B. `EOF`

C. 1

D. `'\\0'`

11. `sprintf(buf, "%d", n);` 的作用是？

A. 从键盘读入

B. 把格式化结果写入字符串 `buf`

C. 打开文件

D. 打印到打印机

## 填空题

12. 输出百分号本身，格式串中应写 `__`。

13. `printf("%05d", 42);` 输出为 `__`。

14. 判断 `scanf("%d", &n)` 是否成功读入一个整数，可检查返回值是否等于 `__`。

15. 循环读取字符直到结束：`while ((ch = getchar()) != __)`。

16. `float` 用 `scanf` 读入对应格式符 `__`；`double` 对应 `__`。

17. `printf("%*d", 4, 7);` 中 `*` 表示宽度由 `__` 指定。

18. 标准输入、标准输出、标准错误对应的流指针分别是 `__`、`__`、`__`。

## 读程序写结果

19. 写出输出：

```
#include <stdio.h>
int main(void) {
 int a = 5;
 double b = 2.5;
 printf("%d %.1f\n", a, b);
 return 0;
}
```

20. 写出输出:

```
#include <stdio.h>
int main(void) {
 printf("%d\n", printf("hi"));
 return 0;
}
```

21. 设输入为:  (数字 3、空格、字母 A) , 写出输出:

```
#include <stdio.h>
int main(void) {
 int n;
 char c;
 scanf("%d", &n);
 scanf(" %c", &c);
 printf("%d%c\n", n, c);
 return 0;
}
```

22. 写出输出:

```
#include <stdio.h>
int main(void) {
 char buf[20];
 sprintf(buf, "%d+%d=%d", 2, 3, 5);
 printf("%s\n", buf);
 return 0;
}
```

23. 写出输出:

```
#include <stdio.h>
int main(void) {
 printf("%-5s|\n", "ab");
 printf("%5s|\n", "ab");
 return 0;
}
```

24. 写出输出:

```
#include <stdio.h>
int main(void) {
 int x = 255;
 printf("%d %x %o\n", x, x, x);
 return 0;
}
```

## 简答或编程

25. 说明 `scanf("%f", &d)` 在 `d` 为 `double` 时为何错误。
26. 编写程序：不断用 `getchar` 读字符并原样 `putchar` 输出，直到遇到 `EOF`。
27. 说明如何处理 `scanf("%d")` 后缓冲区残留换行，再读字符的问题。
28. 编写程序：读入两个整数，按“和 = a + b”的格式输出（如 和 = 3 + 4 且下一行输出结果）。
29. 简述 `printf` 返回值的含义。
30. 编写程序：用 `fgets` 读入一行，去掉末尾换行后用 `puts` 输出。

## 二、参考答案与解析

1. 答案：B  
解析：`int` 用 `%d`。
2. 答案：B  
解析：`scanf` 读 `double` 用 `%lf`；须传 `&x`。
3. 答案：B  
解析：数组名退化为指针。
4. 答案：B  
解析：所有 `unsigned char` 值 + `EOF` 需用 `int` 容纳。
5. 答案：A  
解析：保留两位小数。
6. 答案：B  
解析：缓冲区内常剩 `'\n'`；`%c` 前加空格或先清缓冲。
7. 答案：B  
解析：`EOF` 为负值宏，键盘重定向同样可用。
8. 答案：B  
解析：`long` 常用 `%ld`。
9. 答案：B  
解析：`putchar` 输出单个字符。
10. 答案：B  
解析：读失败且未赋值时常见返回 `EOF`。
11. 答案：B  
解析：`sprintf` 写到字符数组而非 `stdout`。
12. 答案：%%  
解析：两个百分号输出一个 `%`。
13. 答案：00042  
解析：宽度 5，不足前导 0。

14. 答案: 1

解析: 成功赋值的项数。

15. 答案: EOF

解析: 标准结束判断。

16. 答案: %f ; %lf

解析: scanf 对 float / double 区分格式。

17. 答案: 参数 (此处为 4)

解析: \* 从参数列表取宽度。

18. 答案: stdin ; stdout ; stderr

解析: 标准流。

19. 答案: 5 2.5

解析: 按格式输出。

20. 答案:

```
hi2
```

解析: 内层 printf 打印 hi 并返回字符数 2, 外层再打印 2。

21. 答案: 3A

解析: 格式前空格跳过空白, 读到 'A'。

22. 答案: 2+3=5

解析: sprintf 写入后再 printf。

23. 答案:

```
ab |
ab|
```

解析: %-5s 左对齐补空格; %5s 右对齐。

24. 答案: 255 ff 377

解析: 十进制、十六进制、八进制。

25. 答案要点: float 对应 %f, double 对应 %lf; 类型与格式不匹配属未定义行为。

26. 参考代码:

```
#include <stdio.h>
int main(void) {
 int ch;
 while ((ch = getchar()) != EOF)
 putchar(ch);
 return 0;
}
```

27. 答案要点: 在 %c 前加空格 (scanf(" %c", &ch)) 跳过空白; 或用 getchar 循环丢弃到换行; 或统一用 fgetc 再解析。

28. 参考要点: scanf("%d%d",&a,&b); printf("和 = %d + %d\n%d\n", a, b, a+b);

29. 答案要点: 成功时返回写出的字符个数; 出错返回负值。

30. 参考要点: `fgets(s,sizeof(s),stdin);` 用 `strcspn` 去 `'\n'`; `puts(s);`

### 三、本章知识点总结

| 类型     | printf                | scanf                    |
|--------|-----------------------|--------------------------|
| int    | <code>%d</code>       | <code>%d + &amp;</code>  |
| float  | <code>%f</code>       | <code>%f + &amp;</code>  |
| double | <code>%f / %lf</code> | <code>%lf + &amp;</code> |
| 字符串    | <code>%s</code>       | <code>%s</code> (数组名)    |
| 注意     | 宽度精度                  | 缓冲区、返回值、EOF              |

## 第 12 章 · 文件操作

### 本章知识点速览

- `FILE *fp = fopen(路径, 模式);` 失败返回 `NULL`, 用完必须 `fclose`
- `"r"` / `"w"` / `"a"` 及带 `+`、二进制 `b` 的含义不同; `"w"` 会清空已有文件
- 字符: `fgetc / fputc`; 行: `fgets / fputs`; 格式: `fprintf / fscanf`
- 二进制块读写: `fread / fwrite`
- 文本模式可能转换换行; 二进制按原始字节

### 一、题目

#### 选择题

1. `fopen` 成功时返回?

A. `0`  
B. `1`  
C. 非空 `FILE *`  
D. 文件描述符一定是 3
2. 模式 `"w"` 打开已存在文件时?

A. 从末尾追加  
B. 清空原内容后写入  
C. 打开失败  
D. 只读
3. 下列哪项在用完文件后必须做?

A. `rewind`

- B. `fclose`
  - C. `fflush` 即可代替关闭
  - D. `remove`
4. 逐字符复制文件直到结束，循环条件常用？
- A. `fgetc(fp) != 0`
  - B. `(ch = fgetc(in)) != EOF`
  - C. `feof` 先判断再读一次即可（无需关心返回值）
  - D. `fp != NULL` 即可读完
5. 文本模式与二进制模式的重要区别之一是？
- A. 二进制不能用 `fprintf`
  - B. 文本模式可能对换行符做平台转换
  - C. 文本不能存数字
  - D. 二进制不能存结构体
6. `"rb"` 表示？
- A. 只读文本
  - B. 只读二进制
  - C. 读写追加
  - D. 清空后写
7. `fseek(fp, 0, SEEK_END);` 的常见用途是？
- A. 关闭文件
  - B. 将位置移到文件末尾（常用于测文件长度）
  - C. 清空文件
  - D. 删除文件
8. 关于 `"a+"`，正确的是？
- A. 只读
  - B. 追加读写；写操作在末尾
  - C. 必然清空原文件
  - D. 只能二进制
9. `fread` 的返回值表示？
- A. 总是字节数

- B. 成功读取的元素个数
  - C. 文件大小
  - D. 剩余字节必为 0
10. 打开不存在的文件用 "r" 时?
- A. 自动创建空文件
  - B. 返回 NULL
  - C. 返回空字符串
  - D. 程序必崩溃

## 填空题

11. 打开失败时应判断 `fp ==` \_\_\_\_\_。
12. 格式化写入文件用函数 \_\_\_\_\_。
13. `fgets(buf, n, fp)` 最多读入 \_\_\_\_\_ 个字符 (再加 `'\0'`, 即最多 `n-1` 个字符)。
14. 将文件位置移到开头可用 `rewind(fp)` 或 `fseek(fp, 0, _____)`。
15. 检测是否读到文件末尾可用 \_\_\_\_\_ (`fp`) (读失败后再判断更可靠)。
16. 二进制写入数组可用 `fwrite(arr, sizeof(int), n, fp);`, 第三个参数是 \_\_\_\_\_。

## 读程序写结果

17. 设 `a.txt` 内容仅为字符 `x` (无换行)。写出输出:

```
#include <stdio.h>
int main(void) {
 FILE *fp = fopen("a.txt", "r");
 if (!fp) return 1;
 int c = fgetc(fp);
 printf("%c\n", c);
 c = fgetc(fp);
 printf("%d\n", c == EOF);
 fclose(fp);
 return 0;
}
```

18. 写出程序创建的文件 `out.txt` 中的文本内容:

```
#include <stdio.h>
int main(void) {
 FILE *fp = fopen("out.txt", "w");
 fprintf(fp, "%d %s\n", 7, "ok");
 fclose(fp);
 return 0;
}
```

19. 逻辑上说明循环执行次数 (`data.txt` 含 3 个字符 `abc`, 无换行):



```
int n = 0, ch;
FILE *fp = fopen("data.txt", "r");
while ((ch = fgetc(fp)) != EOF) n++;
/* n 为多少? */
```

20. 设 `num.txt` 内容为 `10 20`，写出输出：

```
#include <stdio.h>
int main(void) {
 FILE *fp = fopen("num.txt", "r");
 int a, b;
 fscanf(fp, "%d%d", &a, &b);
 printf("%d\n", a + b);
 fclose(fp);
 return 0;
}
```

21. 写出输出（先写入再读回）：

```
#include <stdio.h>
int main(void) {
 FILE *fp = fopen("t.txt", "w+");
 fputs("Hi", fp);
 rewind(fp);
 char buf[8] = {0};
 fgets(buf, sizeof(buf), fp);
 printf("%s\n", buf);
 fclose(fp);
 return 0;
}
```

## 简答或编程

22. 编写程序：将 `src.txt` 复制为 `dst.txt`（字符流复制），注意检查 `fopen` 与 `fclose`。
23. 编写程序：从键盘读入姓名（不含空格）与成绩，追加写入 `score.txt`（每行一条）。
24. 简述 `"a"` 与 `"w"` 的区别。
25. 编写程序：用二进制方式把一个 `int` 数组写入 `arr.bin`，再读回并打印（示意即可）。
26. 编写程序：统计文本文件行数（以读到的 `'\n'` 个数为准，可说明末行无换行时的处理思路）。
27. 简述为何读写结构体数组到文件时常用二进制 `fread / fwrite`，并指出可移植性注意点。
28. 编写函数示意：`long file_size(FILE *fp);` 用 `fseek / ftell` 测当前文件长度（假设可定位），测完恢复原位置。
29. 说明 `feof` 使用上的常见误区（为何不宜写成 `while (!feof(fp))` 作为主循环条件）。
30. 编写程序：把键盘输入的多行文本写入 `note.txt`，直到读入单独一行 `END` 结束（`END` 本身不写入）。

## 二、参考答案与解析

### 1. 答案：C

解析：成功返回文件指针，失败 `NULL`。

### 2. 答案：B

解析：写模式截断文件。

3. 答案: B

解析: 关闭以刷新缓冲并释放资源。

4. 答案: B

解析: 以 `fgetc` 返回值判 `EOF` 最稳妥。

5. 答案: B

解析: Windows 等上文本模式可能转换 `\n`。

6. 答案: B

解析: `r` 读, `b` 二进制。

7. 答案: B

解析: 定位到末尾后 `ftell` 可得长度 (对可定位文件)。

8. 答案: B

解析: `a+` 为追加更新模式。

9. 答案: B

解析: 返回成功读入的对象个数。

10. 答案: B

解析: 只读打开不存在文件失败。

11. 答案: `NULL`

解析: 打开失败标志。

12. 答案: `fprintf`

解析: 文件版 `printf`。

13. 答案: `n-1`

解析: 预留结束符位置。

14. 答案: `SEEK_SET`

解析: 相对文件开头偏移 0。

15. 答案: `feof`

解析: 通常在读操作失败后调用更准确。

16. 答案: 元素个数 `n`

解析: `fwrite` 第三参为要写的元素个数。

17. 答案:

```
X
1
```

解析: 第二次读到达末尾, `EOF`。

18. 答案: 文件内容为 `7 ok` 后接换行

解析: `fprintf` 按格式写入。

19. 答案: `n == 3`

解析: 读到 a、b、c 各一次。

20. 答案: `30`

解析: `10+20`。

21. 答案: `Hi`

解析: `w+` 读写; `rewind` 后读回。

22. 参考要点: 双 `fopen`; `while ((ch=fgetc(in))!=EOF) fputc(ch,out);`; 分别 `fclose`。
23. 参考要点: `fopen("score.txt","a"); scanf / fprintf(fp,"%s %d\n", name, score);`。
24. 答案要点: `"w"` 清空写; `"a"` 在末尾追加, 不擦除原有内容。
25. 参考要点: `fopen(...,"wb") + fwrite`; 再 `"rb" + fread`; 检查返回的元素个数。
26. 参考要点: 循环 `fgetc`, 遇 `'\n'` 计数; 若文件非空且最后一字符不是换行, 行数可再加 1。
27. 答案要点: 按字节块原样读写效率高、实现简单; 注意结构体对齐/填充、端序、类型大小在不同平台可能不同, 文本格式更易移植。
28. 参考要点: 保存 `ftell`; `fseek(fp,0,SEEK_END)`; `sz=ftell(fp)`; `fseek` 回原位置; 返回 `sz`。
29. 答案要点: `feof` 只在已经试图读过末尾后才置位; `while(!feof)` 往往多循环一次, 应检查读写函数返回值。
30. 参考要点: `fopen("note.txt","w");` 循环 `fgets`; 若行内容为 `END\n` 或去换行后为 `END` 则 `break`; 否则 `fputs`。

### 三、本章知识点总结

| 模式           | 含义                            |
|--------------|-------------------------------|
| r / rb       | 读; 不存在则失败                     |
| w / wb       | 写; 不存在创建, 存在清空                |
| a / ab       | 追加                            |
| r+ / w+ / a+ | 读写变体                          |
| 收尾           | 查 NULL; 必 <code>fclose</code> |

## 第 13 章 · 动态内存

### 本章知识点速览

- `malloc` / `calloc` / `realloc` / `free` 在堆上管理内存 (`stdlib.h`)
- 分配后必须检查是否为 `NULL`; `calloc` 会清零, `malloc` 不会
- 每个成功分配应有配对的 `free`; `free` 后置 `NULL`, 防野指针/重复释放
- `realloc` 失败时原指针仍有效; 成功后应用返回的新指针
- 泄漏: 无法再访问却未 `free`; 悬空: `free` 后继续使用

### 一、题目

#### 选择题

1. `malloc(10 * sizeof(int))` 失败时返回?
- A. 空字符串
- B. `NULL`

C. -1

D. 未初始化的栈地址

2. 与 `malloc` 相比, `calloc(n, size)` 的特点是?

A. 更快且不初始化

B. 将分配的内存置 0

C. 不能分配数组

D. 不用 `free`

3. 下列哪项会造成内存泄漏?

A. `malloc` 后正确 `free`

B. `malloc` 后指针丢失且未 `free`

C. 使用栈数组

D. `calloc` 后 `free`

4. `free(p)` 之后继续写 `*p = 1` 属于?

A. 合法

B. 悬空指针/未定义行为

C. 自动重新分配

D. 编译器保证安全

5. `realloc(p, newSize)` 失败时?

A. 原内存被释放

B. 返回 `NULL`, 原块通常仍有效

C. `p` 自动变 `NULL`

D. 程序必崩溃

6. 释放后为避免误用, 推荐?

A. 再 `malloc` 一次即可不管

B. `p = NULL;`

C. `p++`

D. `free(p)` 两次

7. 关于堆与栈, 下列正确的是?

A. `malloc` 分配在栈上

B. 局部数组通常在栈上, `malloc` 在堆上

C. 堆上对象离开作用域自动释放

D. `free` 用于释放栈变量

8. `free(NULL);` 的行为通常是?

A. 未定义, 必崩溃

B. 安全, 什么也不做

C. 释放全部堆

D. 编译错误

9. 动态二维数组常见做法是?

A. 只能用变长数组, 不能 `malloc`

B. 先分配行指针数组, 再为每行 `malloc`

C. 一次 `malloc` 后不能模拟二维

D. 必须用全局数组

10. 对同一指针连续 `free` 两次 (未置空) 通常属于?

A. 良好习惯

B. 未定义行为

C. 等价于 `realloc`

D. 编译器自动忽略第二次

## 填空题

11. 动态内存头文件是 `#include <_____>`。

12. 推荐: `int *a = malloc(n * _____);`

13. `malloc` 与 `free` 应 \_\_ 使用 (配对)。

14. 对链表每个结点都要 \_\_, 不能只释放头指针 (除非先遍历保存后继)。

15. `realloc` 成功后, 应使用 \_\_ 的返回值作为新指针 (可能与原地址不同)。

16. `sizeof(*p)` 在 `p` 为 `int *` 时等于 \_\_。

## 读程序写结果

17. 写出输出:

```
#include <stdio.h>
#include <stdlib.h>
int main(void) {
 int *p = (int *)malloc(sizeof(int));
 if (!p) return 1;
 *p = 42;
 printf("%d\n", *p);
 free(p);
 return 0;
}
```

18. 写出输出:

```
#include <stdio.h>
#include <stdlib.h>
int main(void) {
 int *p = (int *)calloc(3, sizeof(int));
 if (!p) return 1;
 printf("%d %d %d\n", p[0], p[1], p[2]);
 free(p);
 return 0;
}
```

19. 说明问题 (不必运行) :

```
int *p = malloc(4 * sizeof(int));
p = malloc(8 * sizeof(int));
free(p);
```

20. 写出输出:

```
#include <stdio.h>
#include <stdlib.h>
int main(void) {
 int *p = (int *)malloc(2 * sizeof(int));
 p[0] = 1; p[1] = 2;
 int *q = (int *)realloc(p, 4 * sizeof(int));
 if (q) { p = q; p[2] = 3; p[3] = 4; }
 printf("%d %d %d %d\n", p[0], p[1], p[2], p[3]);
 free(p);
 return 0;
}
```

21. 写出输出:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
int main(void) {
 char *s = (char *)malloc(6);
 if (!s) return 1;
 strcpy(s, "hello");
 printf("%zu\n", strlen(s));
 free(s);
 return 0;
}
```

## 简答或编程

22. 简述野指针、空指针、内存泄漏的区别（各一句话）。
23. 编写函数：动态分配 `n` 个 `int`，读入后返回指针；失败返回 `NULL`（调用者负责 `free`）。
24. 动态创建含 3 个结点的链表（值 1、2、3），遍历打印后正确释放全部结点。
25. 写出正确使用 `realloc` 扩容的推荐写法（用临时指针接返回值，失败时保留原块）。
26. 编写程序：动态分配字符串缓冲区，用 `fgets` 读入一行，打印长度后释放。
27. 简述为何 `int *p = malloc(n * sizeof(int));` 比写死 `malloc(n * 4)` 更可移植。
28. 实现动态数组“追加一个元素”的示意：容量满时 `realloc` 扩为 2 倍再写入。
29. 说明重复释放、越界写、忘记检查 `NULL` 三类错误的危害。
30. 编写函数：释放整条单向链表 `void free_list(Node *head);`。

## 二、参考答案与解析

### 1. 答案：B

解析：堆分配失败返回 `NULL`。

### 2. 答案：B

解析：`calloc` 清零。

### 3. 答案：B

解析：无法 `free` 即泄漏。

### 4. 答案：B

解析：释放后指针悬空。

### 5. 答案：B

解析：失败不释放原块，应保留原指针。

### 6. 答案：B

解析：置空防止二次 `free` 与误用。

### 7. 答案：B

解析：栈自动管理局部对象；堆需手动 `free`。

### 8. 答案：B

解析：标准规定 `free(NULL)` 无操作。

9. 答案：B

解析：常见“指针数组 + 每行一块”或一次分配后手工二维下标映射。

10. 答案：B

解析：重复释放是严重错误。

11. 答案： `stdlib.h`

解析：动态内存函数声明于此。

12. 答案： `sizeof(int)`

解析：按类型计算字节数。

13. 答案：配对 / 一一对应

解析：谁分配谁释放。

14. 答案： `free` (逐个释放)

解析：否则其余结点泄漏。

15. 答案： `realloc`

解析：可能搬家。

16. 答案： `sizeof(int)` (常见 4)

解析：按所指向对象大小。

17. 答案： 42

解析：写入后打印再释放。

18. 答案： 0 0 0

解析： `calloc` 初始化为 0。

19. 答案：第一次分配的内存泄漏 (指针被覆盖且未 `free`)

解析：应先 `free` 旧块或保存指针。

20. 答案： 1 2 3 4

解析：扩容后保留原数据并写入新元素 (假定 `realloc` 成功)。

21. 答案： 5

解析： `"hello"` 长度为 5。

22. 答案要点：野指针：未初始化或已释放仍使用；空指针：值为 `NULL`；泄漏：堆块丢失未释放。

23. 参考：



```
int *read_n(int n) {
 int *a = (int *)malloc(n * sizeof(int));
 if (!a) return NULL;
 for (int i = 0; i < n; i++)
 if (scanf("%d", &a[i]) != 1) { free(a); return NULL; }
 return a;
}
```

24. **参考要点：**三次 `malloc` 链起来；`while (p){ print; q=p->next; free(p); p=q; }`。

25. **参考：**

```
int *t = realloc(p, new_n * sizeof *p);
if (!t) { /* 处理失败, p 仍有效 */ }
else p = t;
```

26. **参考要点：** `char *buf = malloc(256);` 检查；`fgets(buf,256,stdin);` 打印 `strlen`；`free(buf)`。

27. **答案要点：** `sizeof(int)` 随平台可能变化；用 `sizeof` 不依赖写死的 4。

28. **参考要点：** 若 `size==cap`，`cap*=2`，`realloc`；再 `a[size++]=x`。

29. **答案要点：** 重复释放/越界 → 未定义行为、可能破坏堆；忽略 `NULL` → 解引用空指针崩溃。

30. **参考：**

```
void free_list(Node *head) {
 while (head) {
 Node *n = head->next;
 free(head);
 head = n;
 }
}
```

### 三、本章知识点总结

| 函数                   | 要点                       |
|----------------------|--------------------------|
| <code>malloc</code>  | 未初始化；查 <code>NULL</code> |
| <code>calloc</code>  | 清零                       |
| <code>realloc</code> | 可能搬家；失败保原块               |
| <code>free</code>    | 配对；后置 <code>NULL</code>  |
| 错误                   | 泄漏、重复释放、悬空、越界            |

## 第 14 章 · 常用库函数

### 本章知识点速览

- `string.h`: `strlen` / `strcpy` / `strcmp` / `strcat` / `strstr` / `memset` 等
- `stdlib.h`: `atoi` / `malloc` 族 / `rand` / `srand` / `qsort` / `exit`
- `math.h`: `sqrt` / `pow` / `fabs` / `ceil` / `floor` (GCC 常需 `-lm`)
- `ctype.h`: `isalpha` / `isdigit` / `toupper` 等, 参数按 `unsigned char` 习惯使用
- 选择函数时注意头文件、返回值与缓冲区安全

## 一、题目

### 选择题

1. `strlen` 声明在哪个头文件?
  - A. `stdio.h`
  - B. `string.h`
  - C. `math.h`
  - D. `ctype.h`
2. `atoi("123")` 的结果是?
  - A. `"123"`
  - B. `123`
  - C. `1.23`
  - D. 编译错误
3. 判断字符是否为数字, 应使用?
  - A. `isalpha(c)`
  - B. `isdigit(c)`
  - C. `isspace(c)`
  - D. `sqrt(c)`
4. `memset(buf, 0, n)` 的作用是?
  - A. 复制字符串
  - B. 将 `buf` 起 `n` 字节设为 0
  - C. 比较内存
  - D. 分配内存
5. 使用 `sqrt` 需要?
  - A. `#include <math.h>` (链接时可能 `-lm`)
  - B. 只含 `stdio.h`
  - C. 只含 `stdlib.h`

D. 无需头文件

6. `srand((unsigned)time(NULL));` 的常见目的是?

A. 停止随机

B. 用当前时间设置随机种子, 使序列不总相同

C. 排序数组

D. 打开文件

7. `exit(0);` 的含义通常是?

A. 继续循环

B. 立即结束程序, 状态码 0 表示成功

C. 仅退出当前函数

D. 释放一个指针

8. `memcpy` 与 `strcpy` 的主要区别是?

A. 完全相同

B. `memcpy` 按字节复制指定长度, 可处理含 `'\0'` 的数据; `strcpy` 遇 `'\0'` 停止

C. `memcpy` 只能复制字符串

D. `strcpy` 更快且更安全

9. `ceil(2.3)` 的结果约为?

A. 2.0

B. 3.0

C. 2.3

D. -2.0

10. 调用 `qsort` 时必须提供?

A. 文件指针

B. 比较函数

C. 随机种子

D. 数学库

11. 下列关于 `assert` 的说法较合理的是?

A. 运行时条件为假则诊断失败 (调试用)

B. 替代所有 `if`

C. 声明在 `string.h`

#### D. 释放内存

### 填空题

12. 在字符串 `s` 中查找子串 `"ab"`，可用 `_____ (s, "ab")`。
13. 浮点绝对值用 `_____ (x)` (`math.h`)。
14. `toupper('a')` 的结果字符是 `__`。
15. `qsort` 位于头文件 `__`。
16. `rand()` 生成伪随机整数，通常配合头文件 `__` 中的 `srand`。
17. `pow(2, 3)` 的结果是 `__` (数学上)。
18. `isspace(c)` 为真时，`c` 可能是空格、制表符或 `__` 等空白字符。

### 读程序写结果

19. 写出输出：

```
#include <stdio.h>
#include <string.h>
int main(void) {
 char s[] = "Hello";
 printf("%zu\n", strlen(s));
 printf("%s\n", strstr(s, "ll"));
 return 0;
}
```

20. 写出输出：

```
#include <stdio.h>
#include <stdlib.h>
int main(void) {
 printf("%d\n", atoi("45cat"));
 printf("%d\n", abs(-9));
 return 0;
}
```

21. 写出输出：

```
#include <stdio.h>
#include <ctype.h>
int main(void) {
 char c = 'A';
 printf("%d %c\n", islower(c) != 0, tolower(c));
 return 0;
}
```

22. 写出输出：

```
#include <stdio.h>
#include <math.h>
int main(void) {
 printf("%.0f %.0f\n", floor(3.7), ceil(3.2));
 return 0;
}
```

23. 写出输出：

```
#include <stdio.h>
#include <string.h>
int main(void) {
 char a[8];
 memset(a, 'x', 3);
 a[3] = '\0';
 printf("%s %d\n", a, (int)strlen(a));
 return 0;
}
```

24. 写出输出：

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
int cmp(const void *p, const void *q) {
 return *(const int *)p - *(const int *)q;
}
int main(void) {
 int a[] = {3, 1, 2};
 qsort(a, 3, sizeof(int), cmp);
 printf("%d %d %d\n", a[0], a[1], a[2]);
 return 0;
}
```

## 简答或编程

25. 列出 `string.h` 中至少四个常用函数及其一句话用途。
26. 编写程序：读入一行字符串，统计其中字母个数（用 `isalpha`）。
27. 说明为何不推荐 `gets`，应改用什么。
28. 编写程序：生成并打印 5 个 `[0,99]` 范围内的伪随机整数（先 `srand`）。
29. 简述 `atoi` 与 `strtol` 的差异（安全性/错误检测角度）。
30. 编写比较函数：供 `qsort` 对 `double` 数组升序排序（注意直接相减可能不准，可用条件返回）。

## 二、参考答案与解析

### 1. 答案：B

解析：字符串函数在 `string.h`。

### 2. 答案：B

解析：转换为整型 123。

3. 答案: B

解析: `isdigit` 判断十进制数字字符。

4. 答案: B

解析: 按字节填充。

5. 答案: A

解析: 数学库声明与链接。

6. 答案: B

解析: 种子不同则 `rand` 序列不同。

7. 答案: B

解析: `exit` 终止整个程序。

8. 答案: B

解析: 按字节块复制 vs 以 `'\0'` 结尾的字符串复制。

9. 答案: B

解析: 向上取整到不小于该数的最小整数。

10. 答案: B

解析: 需用户提供元素比较规则。

11. 答案: A

解析: `assert.h` 中的断言, 多用于调试。

12. 答案: `strstr`

解析: 返回首次出现位置指针。

13. 答案: `fabs`

解析: `abs` 用于 `int`。

14. 答案: A

解析: `toupper` 将小写转为大写。

15. 答案: `stdlib.h`

解析: 通用工具库。

16. 答案: `stdlib.h` (`time.h` 提供 `time`)

解析: `rand` / `srand` 在 `stdlib.h`; 常用 `time(NULL)` 作种子。

17. 答案: 8

解析: 2 的 3 次方。

18. 答案: 换行符等

解析: 空白字符含 `\n`、`\t`、空格等。

19. 答案:

```
5
11o
```

解析: 长度为 5; `strstr` 返回从 11 起的子串。

20. 答案:

```
45
9
```

解析: `atoi` 转换前导数字; `abs` 取绝对值。

21. 答案: `0 a`

解析: `'A'` 非小写; `tolower` 得 `'a'`。

22. 答案: `3 4`

解析: `floor` 向下; `ceil` 向上。

23. 答案: `xxx 3`

解析: 前三字节为 `x`, 再补结束符。

24. 答案: `1 2 3`

解析: `qsort` 升序。

25. 答案示例: `strlen` 求长; `strcpy` 复制; `strcmp` 比较; `strcat` 连接; (可选 `strstr` / `memset`)。

26. 参考要点: `fgets` 读入; 遍历 `isalpha((unsigned char)s[i])` 计数。

27. 答案要点: `gets` 无法限制长度, 易缓冲区溢出; 改用 `fgets(buf, size, stdin)`。

28. 参考要点:

```
srand((unsigned)time(NULL));
for (int i = 0; i < 5; i++)
 printf("%d ", rand() % 100);
```

29. 答案要点: `atoi` 几乎不报告错误; `strtol` 可通过 `endptr`、`errno` 检测转换范围与是否读到有效数字, 更适合健壮解析。

30. 参考要点:

```
int cmp_d(const void *p, const void *q) {
 double a = *(const double *)p, b = *(const double *)q;
 return (a > b) - (a < b);
}
```

### 三、本章知识点总结

| 头文件      | 常考                    |
|----------|-----------------------|
| string.h | 长度/复制/比较/查找/memset    |
| stdlib.h | 转换、内存、随机、qsort、exit   |
| math.h   | sqrt/pow/fabs; 可能 -lm |
| ctype.h  | is* / to* 字符分类转换      |
| 安全       | 弃用 gets; 注意缓冲区与返回值    |